

RISC-V Hardware–Software Co-Design Methodology

Prof. Dr. Tassadaq Hussain

From workload requirements to an integrated, measurable and verifiable
accelerator-enabled SoC.

Learning Objectives

BY THE END OF THIS LECTURE

Students should be able to move from a workload to a justified RISC-V system architecture.

Co-design principles

Explain hardware–software co-design and the system architecture flow.

Accelerator integration

Select an appropriate strategy for connecting an accelerator to RISC-V.

Design trade-offs

Balance performance against area, power and resource budgets.

SoC reasoning

Apply system-level considerations through worked case studies.

Agenda

The lecture follows the same sequence used in a real co-design project.

Co-design foundations

System architecture methodology

Accelerator architecture and RISC-V integration

SoC, memory and system software

Performance, area and power trade-offs

Implementation frameworks and final workflow

Start

Define the problem and establish the software baseline.

Design

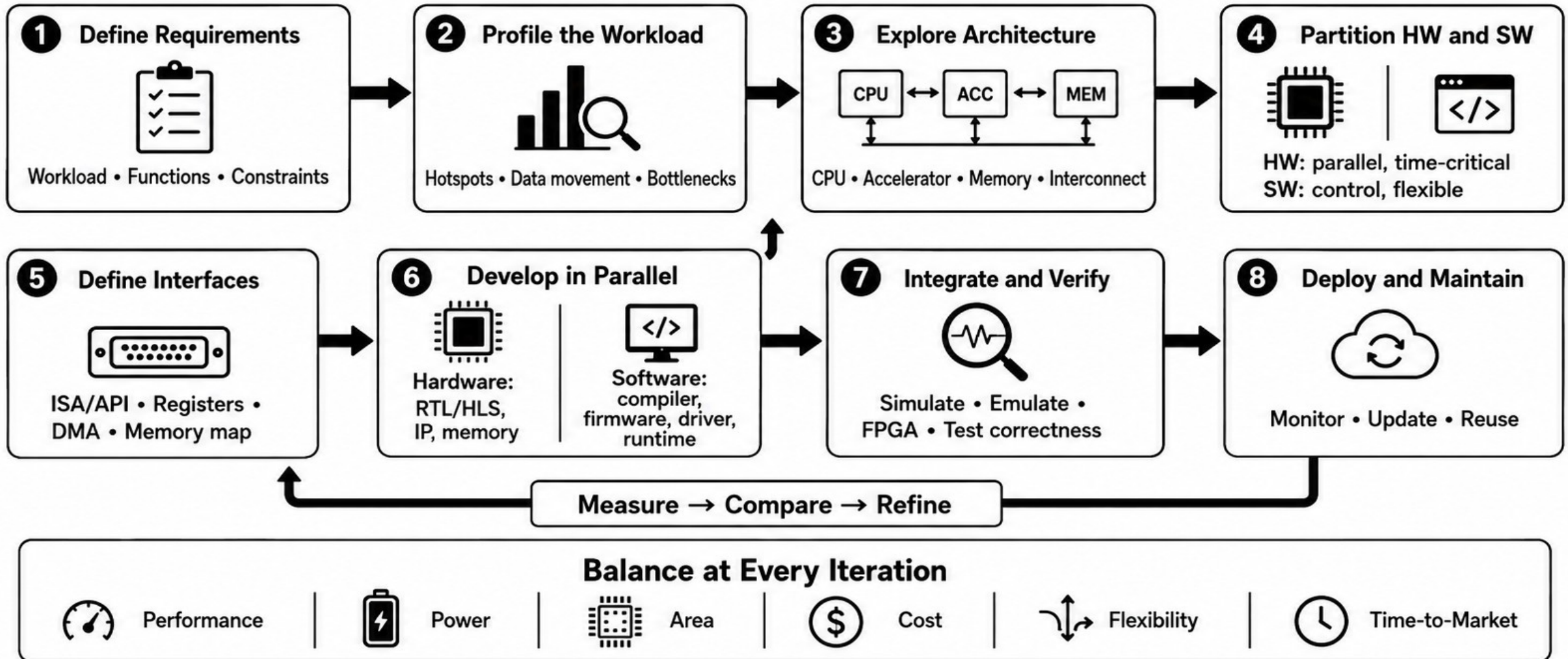
Partition, integrate and model the complete system.

Decide

Measure alternatives and select a defensible architecture.

The First Co-Design Question

What problem must the complete system solve—and what measurable constraints define success?



Why Hardware–Software Co-Design?

SYSTEM LIMITS ARE CONNECTED

Improving one component in isolation can simply move the bottleneck elsewhere.



Performance

Specialized datapaths can remove expensive repeated work from the CPU.



Energy

Purpose-built hardware can perform useful operations with less movement and control overhead.



System fit

Software, memory and interconnect choices determine whether hardware gains are usable.

CPUs Provide Flexibility; Accelerators Provide Efficiency

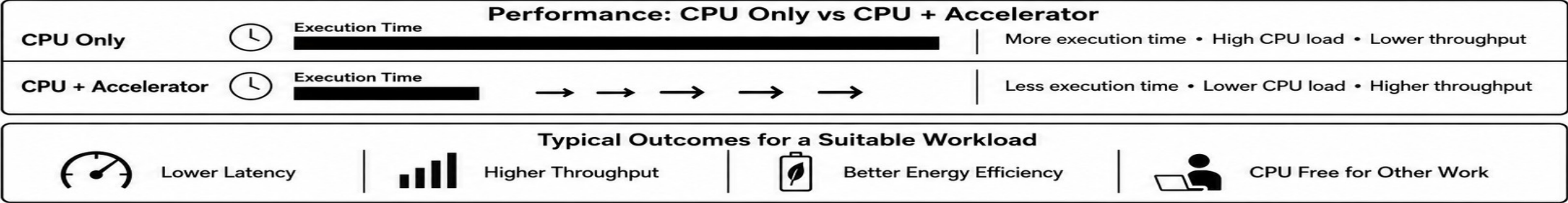
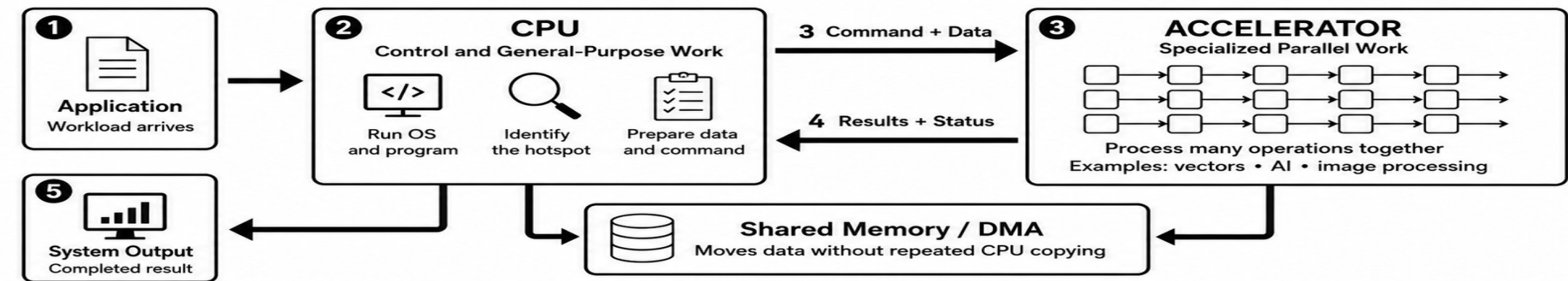
The design challenge is to combine both without losing programmability, correctness or system balance.

General-purpose CPU

- Runs diverse control-oriented software
- Mature compiler and operating-system support
- Fast to change after deployment
- Pays fetch, decode and scheduling overhead

Hardware accelerator

- Optimized for a focused computation
- Exploits data, task or pipeline parallelism
- Can improve performance per watt
- Requires interfaces, data movement and software support



Co-Design Optimizes the Complete System

CONCURRENT DEVELOPMENT

Hardware and software decisions are refined together instead of being handed off in isolation.



Hardware view

Datapath, storage, interfaces, timing and physical resource choices.



Software view

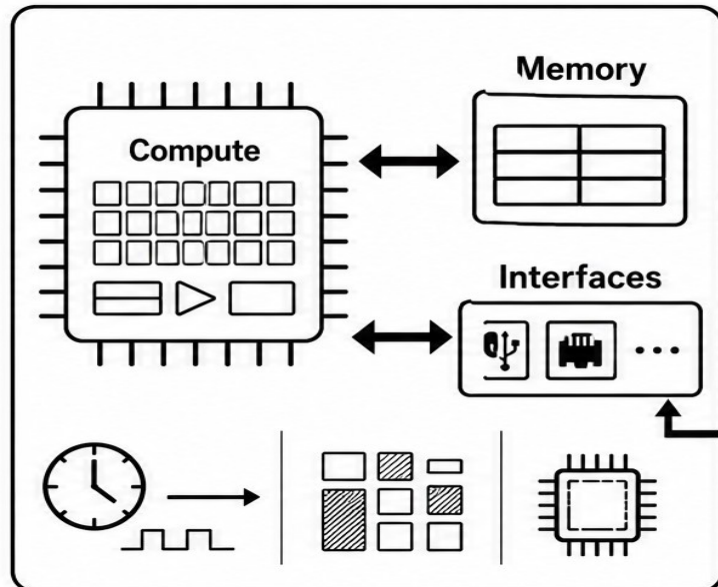
Algorithms, drivers, runtime, compiler support and application behavior.



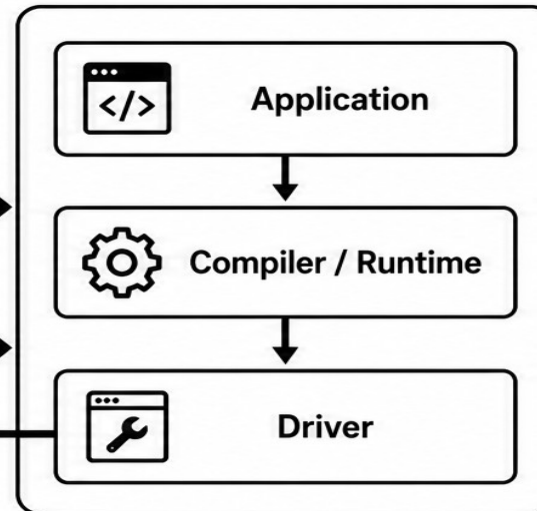
System view

End-to-end correctness, performance, energy, cost and maintainability.

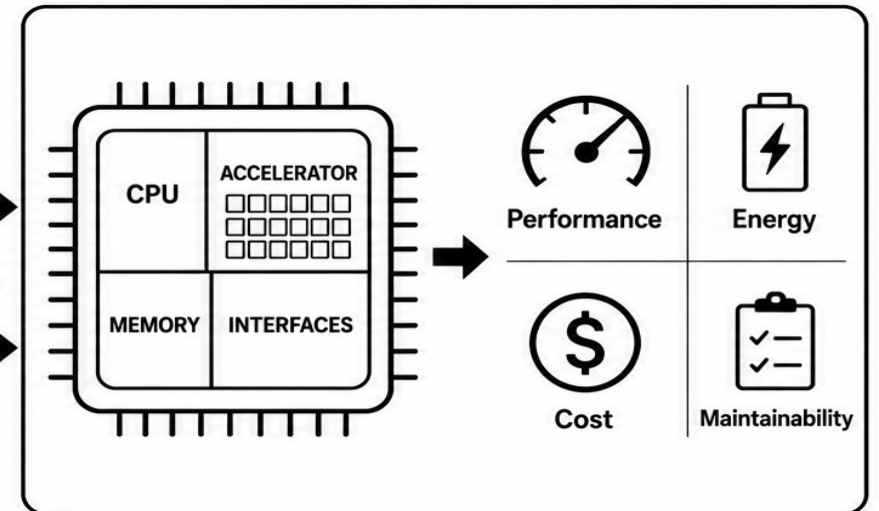
HARDWARE VIEW



SOFTWARE VIEW



SYSTEM VIEW



Every Co-Design Decision Connects Multiple Layers

01	Application	User-visible function and workload
02	Algorithm	Operations, data dependencies and precision
03	Software	Libraries, runtime, driver and compiler
04	ISA interface	Instructions, CSRs, memory and synchronization
05	Hardware	Datapath, control and local storage
06	SoC	Interconnect, memory, I/O, clocks and protection

Hardware and Software Must Have Explicit Responsibilities

A clean boundary reduces integration risk and keeps the system understandable.

Hardware owns

- Time-critical repeated computation
- Cycle-level data movement and control
- Parallel execution and local buffering
- Status, error and performance counters

Software owns

- Configuration and sequencing
- Memory allocation and data preparation
- Scheduling, synchronization and recovery
- User-facing API and system policy

Begin with the Workload and Use Case

The same kernel may require a different architecture in a sensor node, embedded Linux system or high-throughput server.

Describe the problem

Input and output data

Workload size and arrival rate

Latency and throughput needs

Operating environment

Expected lifetime and update model

Function

What result must the system produce?

Context

Where and how often will it operate?

Constraint

What limits cannot be exceeded?

A Design Is Successful Only Against a Measured Baseline

Reference implementation

BASELINE FIRST

Run the complete workload on the target RISC-V CPU before proposing specialized hardware.

Acceptance rule

The accelerator must improve the whole application—not only the isolated kernel.

Required improvement

Set a minimum latency, throughput, speedup or energy target that matters at system level.

Hard limits

Record area, power, memory, bandwidth, cost and development constraints.

Agenda

The lecture follows the same sequence used in a real co-design project.

Co-design foundations

System architecture methodology

Accelerator architecture and RISC-V integration

SoC, memory and system software

Performance, area and power trade-offs

Implementation frameworks and final workflow

Co-Design Is an Iterative Flow from Requirements to Evidence

- | | | |
|-----------|------------------|--|
| 01 | Specify | Capture function, workload and constraints |
| 02 | Measure | Establish the CPU baseline and profile |
| 03 | Partition | Assign functions to hardware and software |
| 04 | Architect | Select compute, memory and interfaces |
| 05 | Implement | Build models, RTL and software |
| 06 | Validate | Compare evidence and refine the design |

Step 1 — Capture Functional Requirements

WHAT THE SYSTEM MUST DO

Functional requirements define observable behavior before implementation choices are made.



Inputs

Data format, ranges, arrival order and valid/ready behavior.



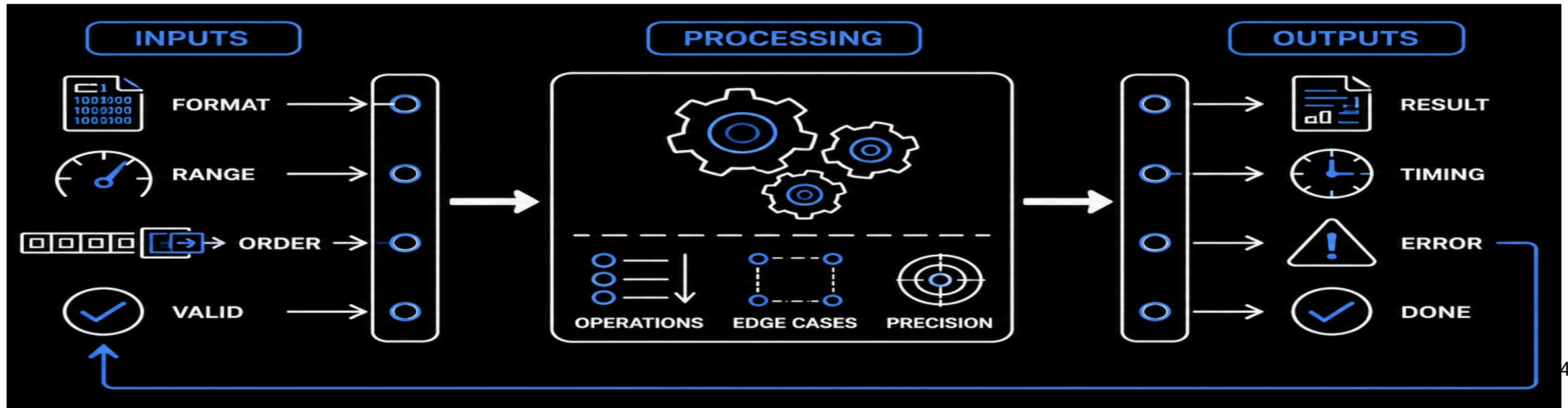
Processing

Required operations, ordering, corner cases and precision rules.



Outputs

Result format, timing, error reporting and completion behavior.



SPECIFY THE ALGORITHM FIRST

1. INPUT CONTRACT



Data format



Range & size

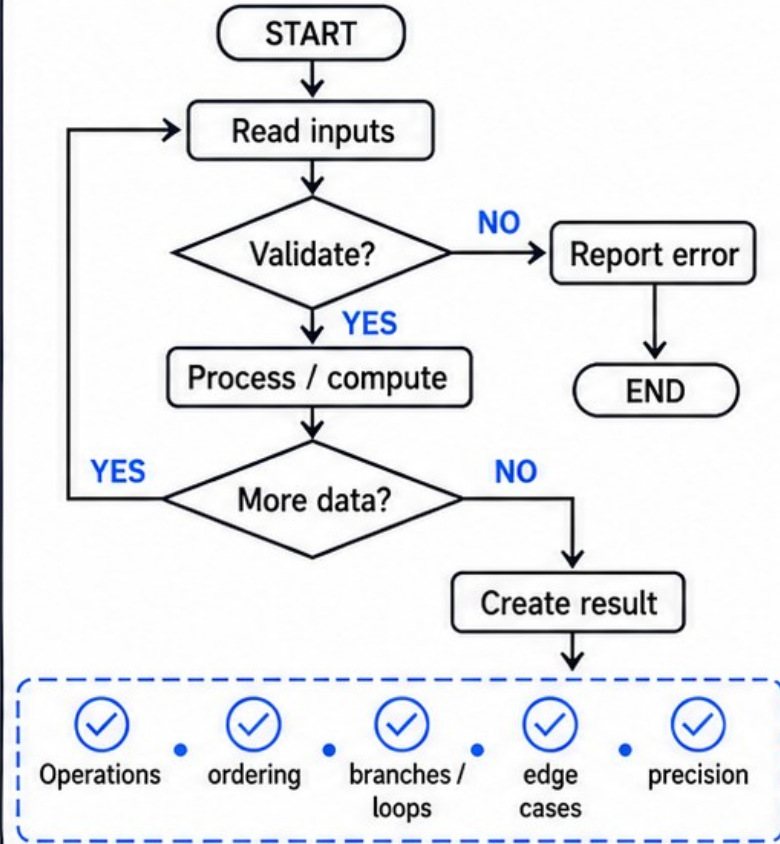


Arrival / order



Valid / invalid

2. ALGORITHM CONTROL FLOW



3. OUTPUT CONTRACT



Result format



Timing

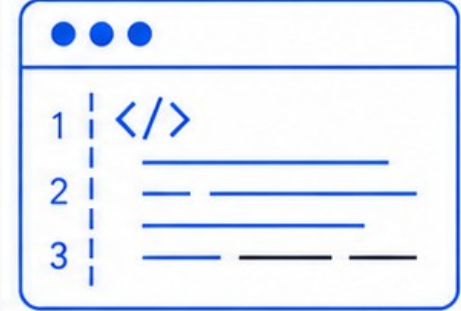


Errors



Completion

4. BASELINE CODE



- ✓ Correct reference implementation
- ✓ Test vectors + expected outputs
- ✓ Measure CPU latency / throughput

Functional control-flow diagram → Baseline reference code

Application Spectrum: Monte Carlo, Vision Models, and Language Models

Monte Carlo / sampling: highly parallel, often compute-heavy if each sample does significant math

Vision models: mixed behavior, dense compute + activation movement

Language models: often bandwidth- and memory-bound, especially in autoregressive inference

Application Spectrum: Monte Carlo, Vision Models, and Language Models

Comparing input/output flow, FLOPS demand, memory behavior, and reuse

1) Monte Carlo / Sampling

Often Compute-Bound



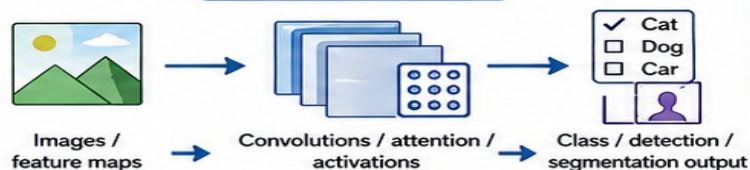
Parallelism	████████
FLOPS demand	████████
Memory traffic	██████
Memory reuse	██████

- Highly parallel: many samples can run independently
- Often compute-heavy if each sample does significant math
- Input pattern: small state per sample
- Output pattern: reduced result or statistics
- Memory reuse: moderate to high when sample state fits in cache
- Optimization focus: parallelism, vectorization, GPU threads

 Typical examples: simulation, risk analysis, particle methods

2) Vision Models

Mixed Behavior



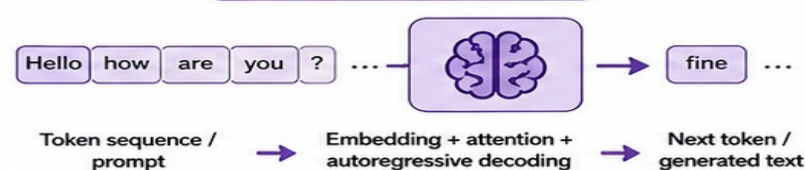
Parallelism	████████
FLOPS demand	████████
Memory traffic	██████
Memory reuse	██████

- Mixed behavior: dense compute plus activation movement
- High FLOPS in matrix/convolution kernels
- Input pattern: large tensors and feature maps
- Output pattern: labels, boxes, masks, embeddings
- Memory reuse: good inside tiled kernels, but activations still move a lot
- Optimization focus: tensor cores, batching, tiling, fused kernels

 Typical examples: CNNs, ViTs, object detection

3) Language Models

Often Memory-Bound



Parallelism	██████
FLOPS demand	██████
Memory traffic	████████
Memory reuse	██████

- Often bandwidth- and memory-bound
- Especially in autoregressive inference
- Input pattern: token stream and large model weights
- Output pattern: one token at a time during generation
- Memory reuse: low to medium during inference because weights and KV cache dominate movement
- Optimization focus: bandwidth, cache locality, quantization, KV-cache efficiency

 Typical examples: chatbots, summarization, code generation

Simple takeaway

More compute-dominated

Mixed compute + data movement

More memory / bandwidth-dominated

Optimization depends on the bottleneck, not only on the algorithm name.

Step 2 — Capture Non-Functional Constraints

HOW WELL THE SYSTEM MUST OPERATE

These constraints shape the architecture even when the required function is unchanged.

Performance

Latency, throughput, deadline and utilization.

Power + energy

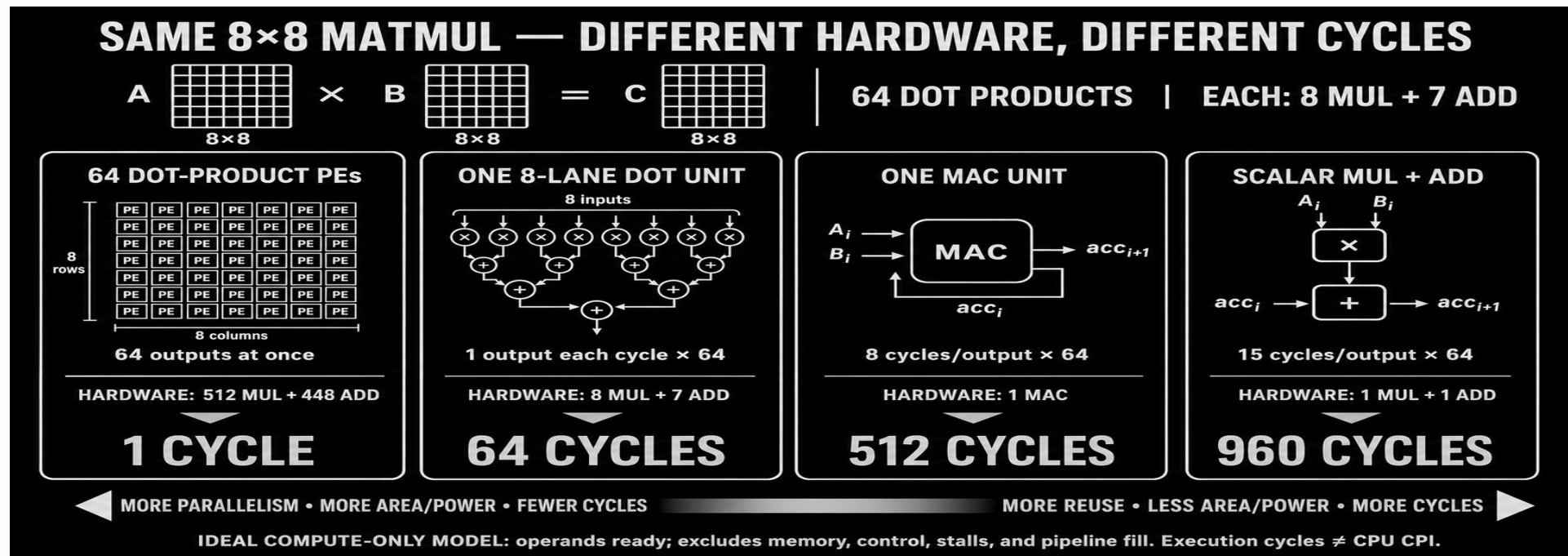
Peak power, average power and energy per task.

Resources

Area, LUTs, DSPs, BRAM, memory and bandwidth.

Engineering

Cost, schedule, verification, portability and maintainability.



$$\text{Execution Time} = \text{Instruction Count} \times \text{CPI} \times \text{Clock Period}$$

Complete workload

MEASURE

Use representative inputs and include initialization, data movement and control overhead.

Purpose

The baseline reveals whether acceleration is necessary and how much improvement is required.

Collect evidence

Record cycles, execution time, memory traffic, cache behavior and CPU utilization.

Preserve correctness

Save reference outputs so every accelerated version can be checked against them.

Step 3 — Profile the Application

Acceleration should target work that materially affects end-to-end execution.

Profile before partitioning

Time by function or kernel

Instruction and operation mix

Cache misses and memory traffic

Branch and stall behavior

Input-dependent variation

Hotspot

Consumes a large share of execution time or energy.

Stability

Appears consistently across representative workloads.

Opportunity

Contains exploitable regularity or parallelism.

Step 4 — Characterize the Candidate Kernel

UNDERSTAND THE WORK BEFORE DESIGNING HARDWARE

Kernel structure determines whether specialization will help and what architecture it needs.

Compute

Operation count, dependency chain and arithmetic intensity.

Memory

Access pattern, reuse, footprint and transfer volume.

Control

Branches, loops, irregularity and exception behavior.

Communication

Arguments, results, synchronization and invocation rate.

Workload Classes: Compute-Bound, Memory-Bound, and I/O-Bound

Compute-bound: limited by arithmetic throughput

- Example: dense numerical kernels with high reuse

Memory-bound: limited by memory bandwidth / cache misses

- Example: element-wise ops, sparse access, autoregressive LLM inference

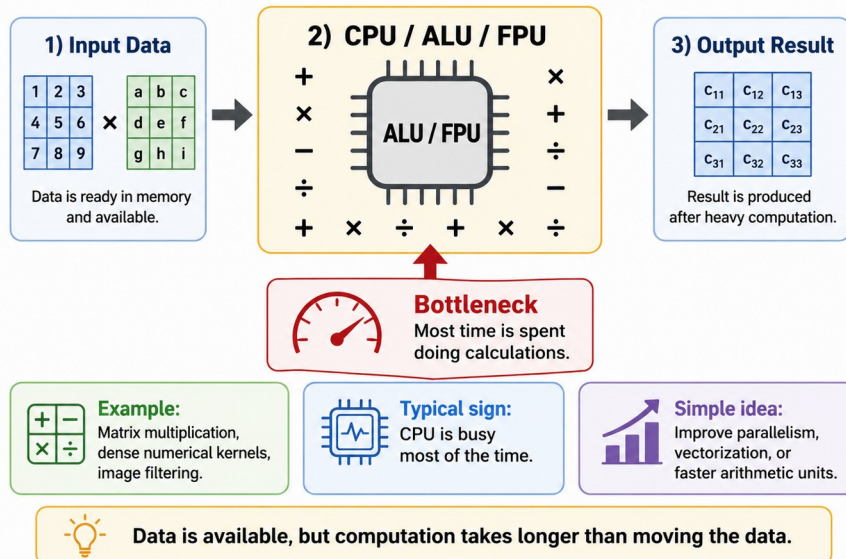
I/O-bound: limited by disk, network, or external device transfer

- Example: data loading, checkpoint loading, streaming pipelines

Optimization depends on the bottleneck, not only on the algorithm name

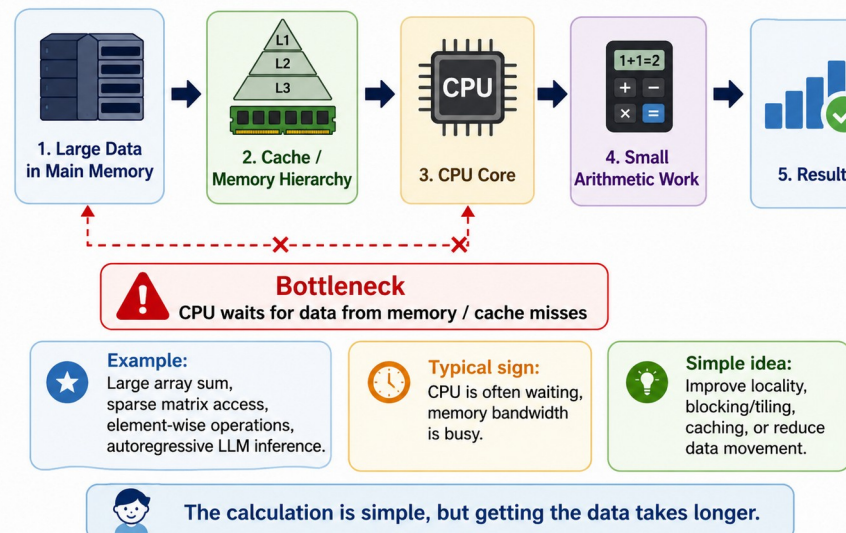
Compute-Bound Application

Performance limited mainly by arithmetic work



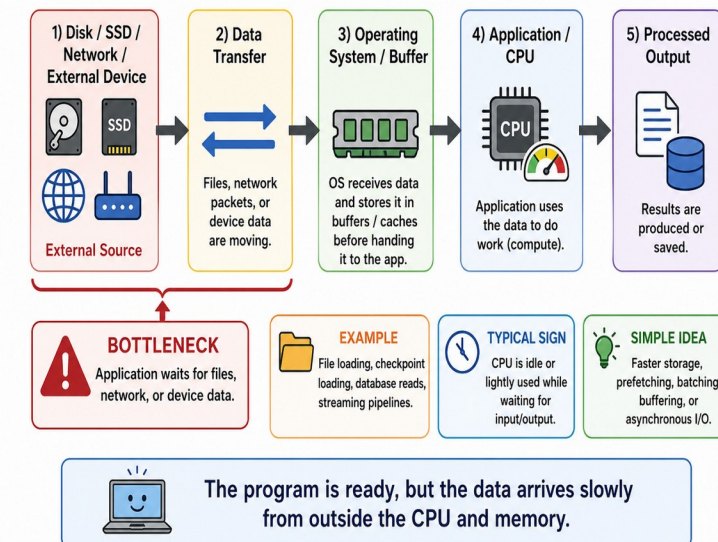
Memory-Bound Application

Performance limited mainly by memory access



I/O-Bound Application

Performance limited mainly by input/output transfer



Expose the Parallelism Available in the Workload

PARALLELISM MUST EXIST BEFORE HARDWARE CAN EXPLOIT IT

Different forms of independence lead to different accelerator structures.

Instruction-level

Independent operations execute in parallel or a deeper pipeline.

Data-level

The same operation processes many elements or lanes.

Task-level

Independent functions or requests execute concurrently.

Pipeline-level

Successive stages operate on different data items.

Expose Parallelism in the C Workload

Independence in the code determines the accelerator structure.

1. C workload

2. Check dependencies

3. Expose independence

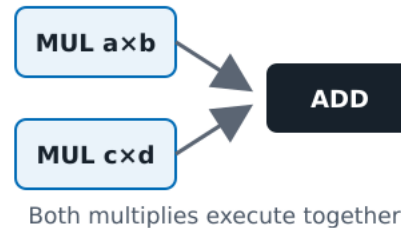
4. Choose parallel hardware

Instruction-level parallelism

INDEPENDENT OPERATIONS

```
int p0 = a * b;  
int p1 = c * d;  
// p0 and p1 are independent  
int y = p0 + p1;
```

PARALLEL DATAPATH



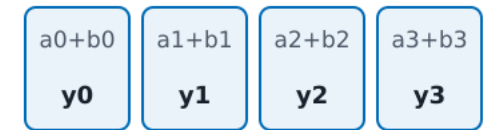
Pipelined / multi-unit datapath

Data-level parallelism

INDEPENDENT LOOP ITERATIONS

```
for (int i=0; i<N; ++i)  
y[i] = a[i] + b[i];  
// same operation, many items
```

VECTOR LANES



Four elements processed in one step

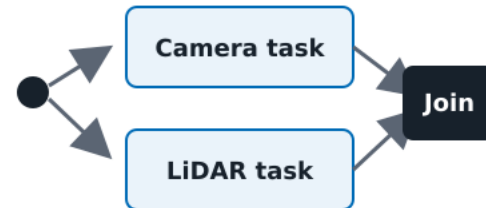
RVV / SIMD / GPU lanes

Task-level parallelism

INDEPENDENT FUNCTIONS / TASKS

```
#pragma omp parallel sections  
{  
#pragma omp section  
process_camera();  
#pragma omp section  
process_lidar();  
}
```

CONCURRENT ENGINES



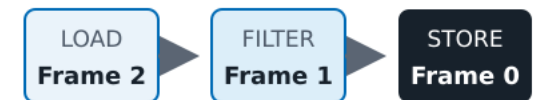
CPU cores / multiple accelerators

Pipeline-level parallelism

SUCCESSIVE STAGES, DIFFERENT ITEMS

```
for (int f=0; f<F; ++f) {  
load(f);  
filter(f);  
store(f);  
}
```

STREAMING PIPELINE — SAME CYCLE



Three frames occupy three stages concurrently

Streaming stages + FIFO buffers

- Parallelism must exist in the workload before hardware can exploit it.

Precision Is an Architectural Decision

The required numerical range and accuracy determine datapath width, memory traffic and resource cost.

Higher precision

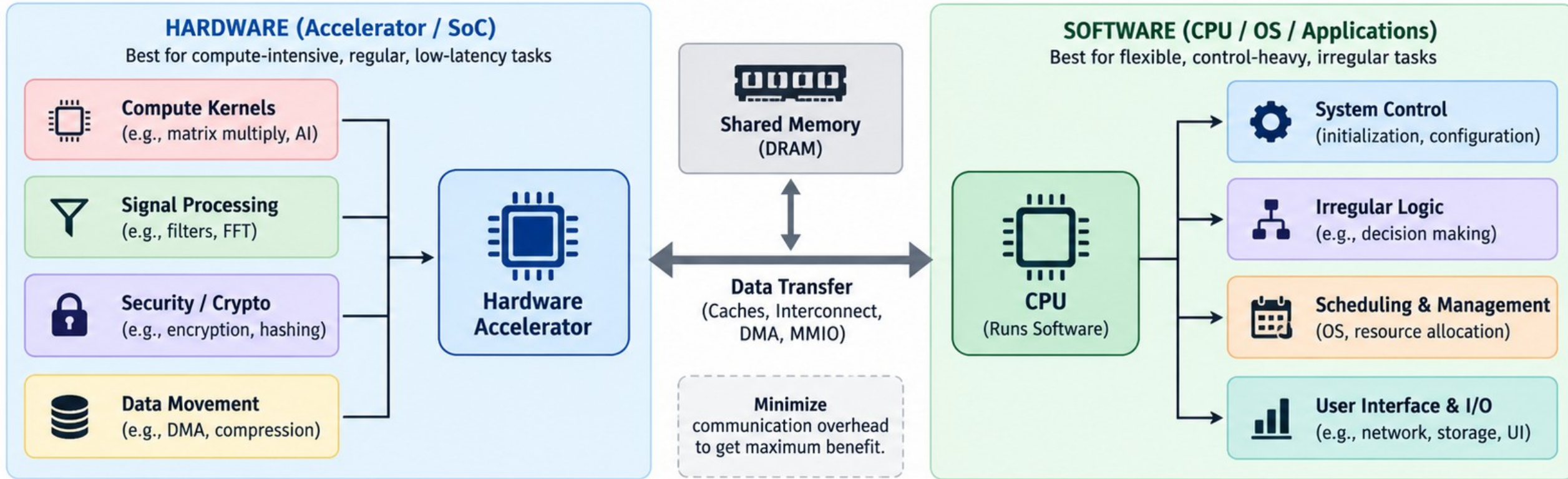
- Greater dynamic range or numerical accuracy
- Wider operators and storage
- More bandwidth per element
- Often higher area and energy

Reduced precision

- Narrower datapaths and buffers
- More parallel elements per resource budget
- Lower transfer volume
- Requires application-level accuracy validation

Step 5 — Partition Functions Across Hardware and Software

* Partitioning balances acceleration benefit against communication and engineering cost.



Keep in Hardware

- Irregular control, low-latency functions
- Functions that change infrequently
- Performance-critical kernels



Move to Hardware

- Stable, repeated and parallel computations
- Measurable performance or power gain
- Justifies hardware cost and complexity



Share Responsibility

- Software configures and schedules
- Hardware executes compute-intensive parts
- Co-design for best overall system impact

Right function, right place — for higher performance, lower cost, and better scalability.

Define the Hardware–Software Contract Before RTL

THE INTERFACE IS PART OF THE ARCHITECTURE

Both sides must agree on how work is described, started, completed and recovered.

Commands

Operation, mode, size, addresses and configuration.



- Operation (e.g., read, write, compute)
- Mode (e.g., blocking, non-blocking)
- Size/length (e.g., bytes, elements)
- Addresses (source, destination)
- Configuration (e.g., control registers)

Example

```
READ 0x8000_0000
SIZE 4 KB
MODE AUTO
START
```

Data

Format, alignment, ownership and lifetime.



- Format (e.g., int8, int32, float)
- Alignment (e.g., 4B, 16B)
- Ownership (who reads/writes)
- Lifetime (validity and reuse)
- Layout (e.g., contiguous, tiled)

Example

```
Format:  int32
Alignment: 16 B
Owner:    Device
Lifetime: Until complete
Layout:   Contiguous
```

Status

Busy state, progress, counters and errors.



- Busy/Idle state
- Progress indication (e.g., % or count)
- Performance counters
- Completion status (success/fail)
- Error codes and error details

Example

```
BUSY: 0
PROGRESS: 75%
SAMPLES: 1,048,576
DONE: 00
ERROR: NONE
```

Synchronization

Polling, interrupt, fences, ordering and timeout behavior.



- Completion mechanism (polling or interrupt)
- Memory ordering (fences/barriers)
- Timeout behavior
- Multiple in-flight commands
- Recovery on timeout or error

Example

```
MODE: Interrupt
FENCE: After write
TIMEOUT: 100 ms
ORDER: In-order
RECOVERY: Reset + retry
```

A clear hardware–software contract enables correct, efficient and maintainable designs.

Agenda

The lecture follows the same sequence used in a real co-design project.

Co-design foundations

System architecture methodology

Accelerator architecture and RISC-V integration

SoC, memory and system software

Performance, area and power trade-offs

Implementation frameworks and final workflow

An Accelerator Combines Datapath, Control and Local Storage

ARCHITECTURE FOLLOWS THE KERNEL

The internal structure should directly reflect the operation graph and data-reuse pattern.



Datapath

Functional units, pipelines, lanes and routing execute operations.



Control

FSMs, counters and handshakes sequence the computation.



Storage

Registers, FIFOs and SRAM buffers retain reusable or in-flight data.

Choose an Execution Architecture That Matches the Workload

ONE STRUCTURE DOES NOT FIT EVERY KERNEL

The best architecture exposes useful parallelism without creating a larger data-movement problem.

Scalar pipeline

Sequential or lightly parallel operations with tight control.

SIMD / vector

Regular element-wise operations over arrays.

Streaming

Continuous data passes through connected processing stages.

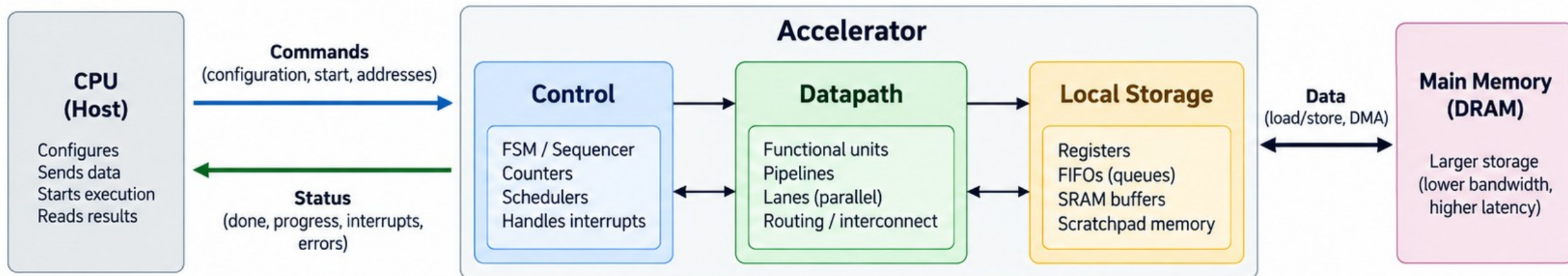
Systolic

Dense, regular reuse across a spatial array of processing elements.

An Accelerator Combines Datapath, Control and Local Storage

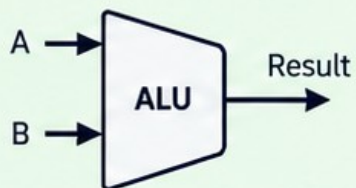
ARCHITECTURE FOLLOWS THE KERNEL

The internal structure should directly reflect the operation graph and data-reuse pattern.



Datapath

Functional units, pipelines, lanes and routing execute operations.

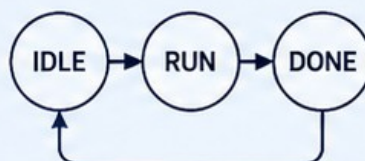


- Performs compute operations
- Can be deeply pipelined
- Multiple lanes for parallelism
- Interconnect routes data between units



Control

FSMs, counters and handshakes sequence the computation.



- Starts and stops operations
- Controls data movement
- Handles handshakes and synchronization
- Manages interrupts and errors



Storage

Registers, FIFOs and SRAM buffers retain reusable or in-flight data.



- Holds input, intermediate and output data
- Enables data reuse (reduces memory traffic)
- Supports high bandwidth and low latency access

A well-designed accelerator matches the kernel's operation graph and data-reuse pattern to achieve higher performance and energy efficiency.

Integration Ranges from an Instruction to an Independent Master

01 Custom instruction

Execute through the processor pipeline

02 Coprocessor

Use a tightly coupled command / response interface

03 MMIO peripheral

Configure through memory-mapped registers

04 DMA accelerator

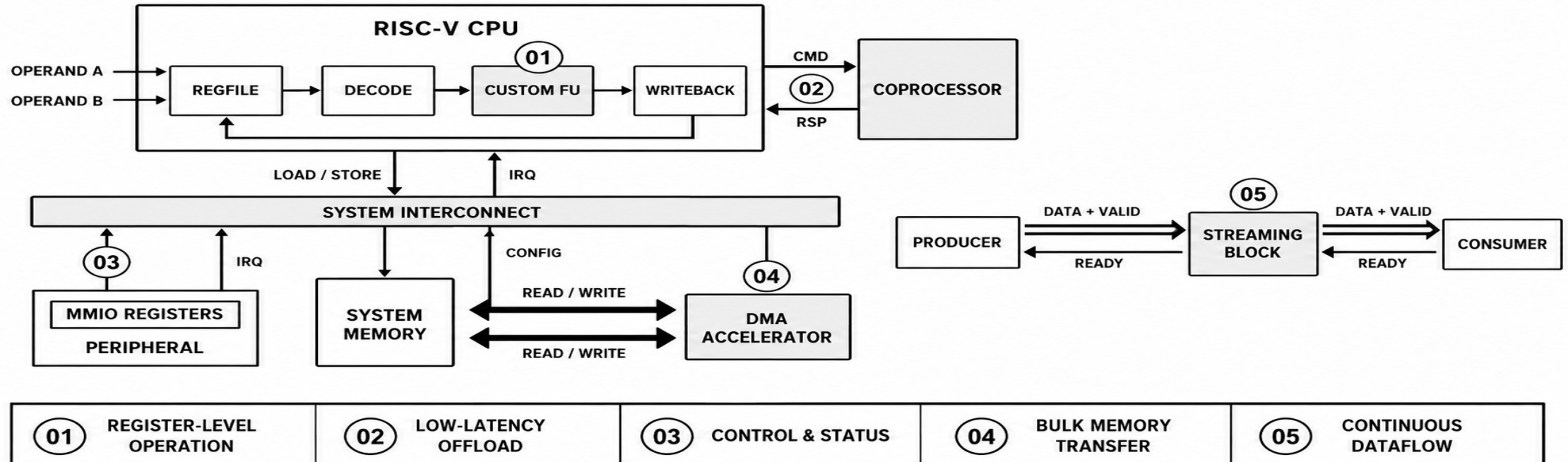
Read and write system memory independently

05 Streaming block

Exchange continuous data through ready / valid links

06 Selection

Match coupling and bandwidth to the workload



Option 1 — Custom RISC-V Instructions

A custom instruction makes a specialized operation part of the processor-visible ISA.

Best suited to

Small operand and result sets

Low-latency frequent operations

Register-based communication

Compiler or intrinsic support

Benefit

Low invocation overhead and natural instruction sequencing.

Cost

Pipeline, decode, toolchain and verification changes.

Avoid when

The operation needs large buffers or long independent execution.

Option 2 — Tightly Coupled Coprocessor

A command / response interface gives specialized hardware close access to the core without redefining the base pipeline.

Typical interaction

Issue an operation and operands

Coprocessor executes

Return a result or completion

Coordinate exceptions and stalls

Benefit

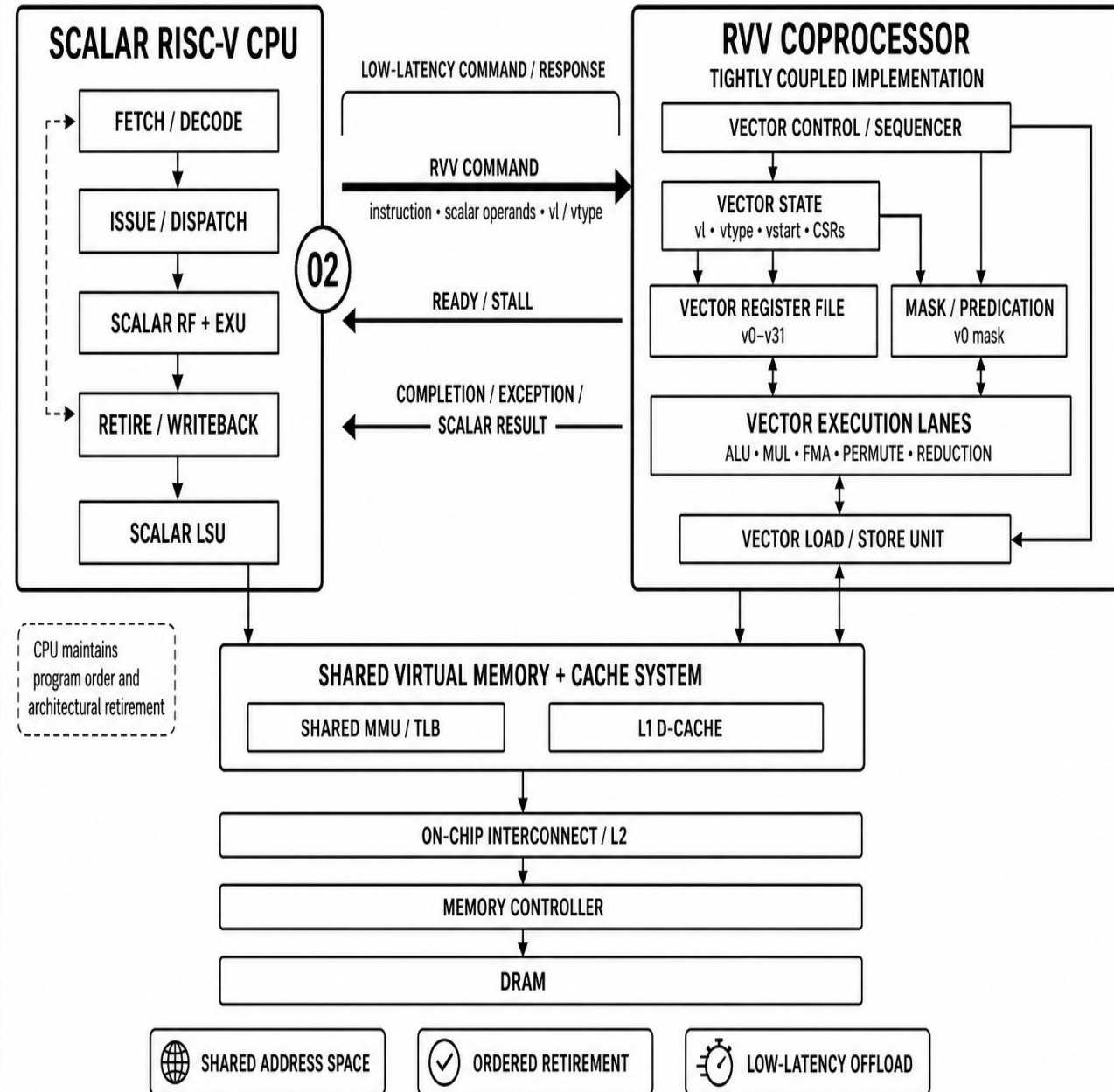
Lower overhead than a distant peripheral.

Trade-off

Greater coupling to the selected processor implementation.

Example pattern

RoCC-like accelerator attachment in a generated RISC-V SoC.



Option 3 — Memory-Mapped Accelerator

CONTROL THROUGH THE ADDRESS SPACE

The CPU treats accelerator registers like other MMIO peripherals.

Configure

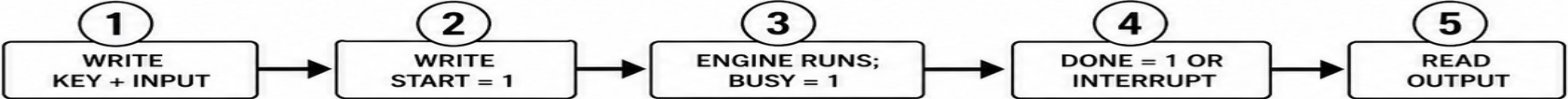
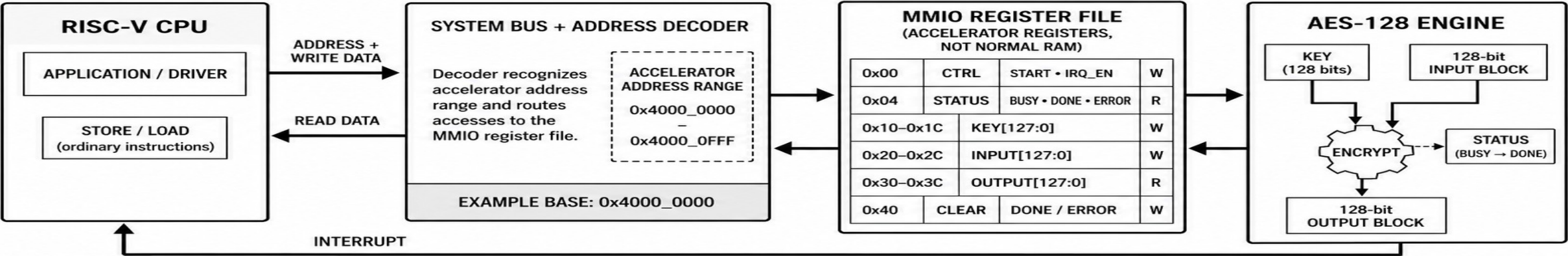
Write operation, addresses, sizes and mode into CSRs.

Start + observe

Set a start bit, then read busy, done or error status.

Complete

Poll status or receive an interrupt before consuming results.



MMIO is best when control is simple and data is small or occasional.

Option 4 — DMA-Capable Bus Master

MOVE BULK DATA WITHOUT CPU COPYING

The accelerator generates memory transactions after software supplies buffer addresses and sizes.

Setup

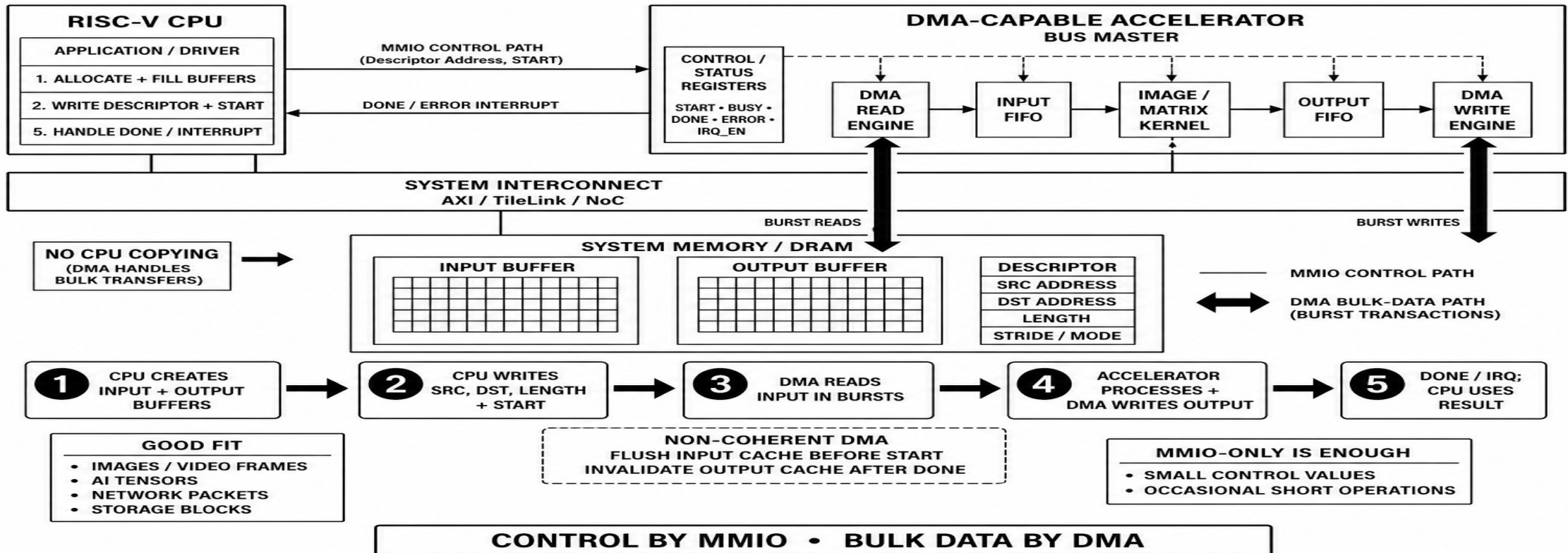
Software allocates buffers, writes descriptors and starts the transfer.

Transfer

The accelerator reads inputs and writes outputs using burst transactions.

Completion

Status or an interrupt returns ownership to software.



Option 5 — Streaming Accelerator

PROCESS DATA AS IT ARRIVES

Ready / valid interfaces decouple stages and avoid assigning an address to every transfer.



Producer

Sensor, DMA engine or upstream accelerator supplies a stream.



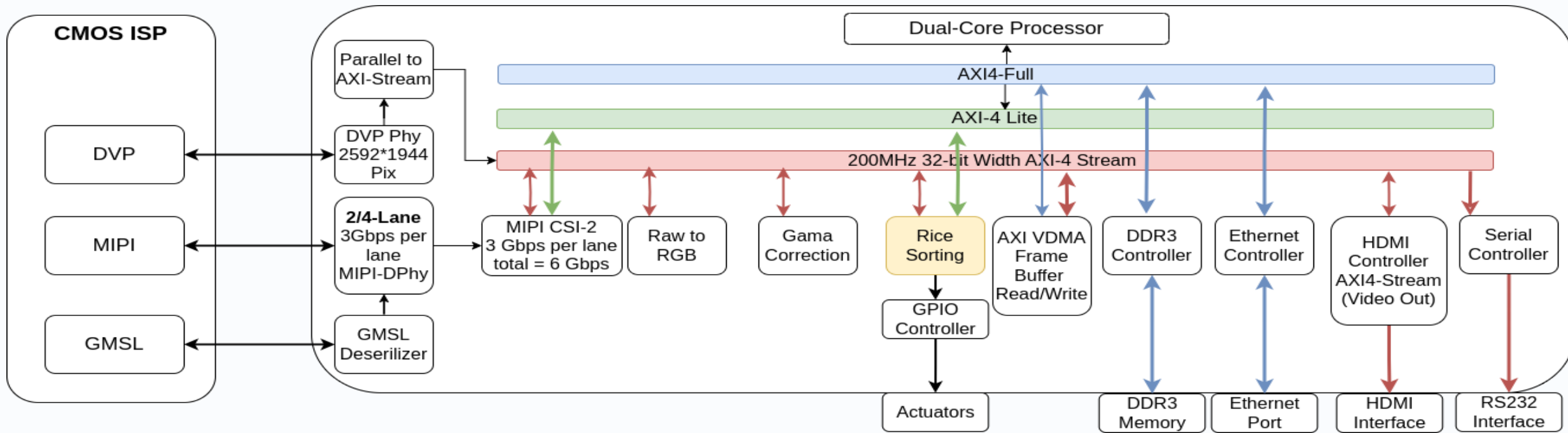
Pipeline

Each stage transforms data and maintains flow control.



Consumer

Downstream logic, memory writer or display accepts output.



Choose Coupling According to Operand Size, Latency and Autonomy

01	Custom instruction	Tiny operands · minimum latency
02	Coprocessor	Close coupling · short operations
03	MMIO	Simple control · moderate invocation rate
04	DMA master	Bulk arrays · autonomous execution
05	Streaming	Continuous data · sustained throughput
06	Hybrid	MMIO control plus DMA or stream data

Protocol Choice Should Follow Traffic and Ecosystem Needs

Different interconnect protocols are optimized for different bandwidth, latency, complexity and ecosystem requirements. Choose the right protocol based on your data movement pattern, performance needs and available ecosystem.

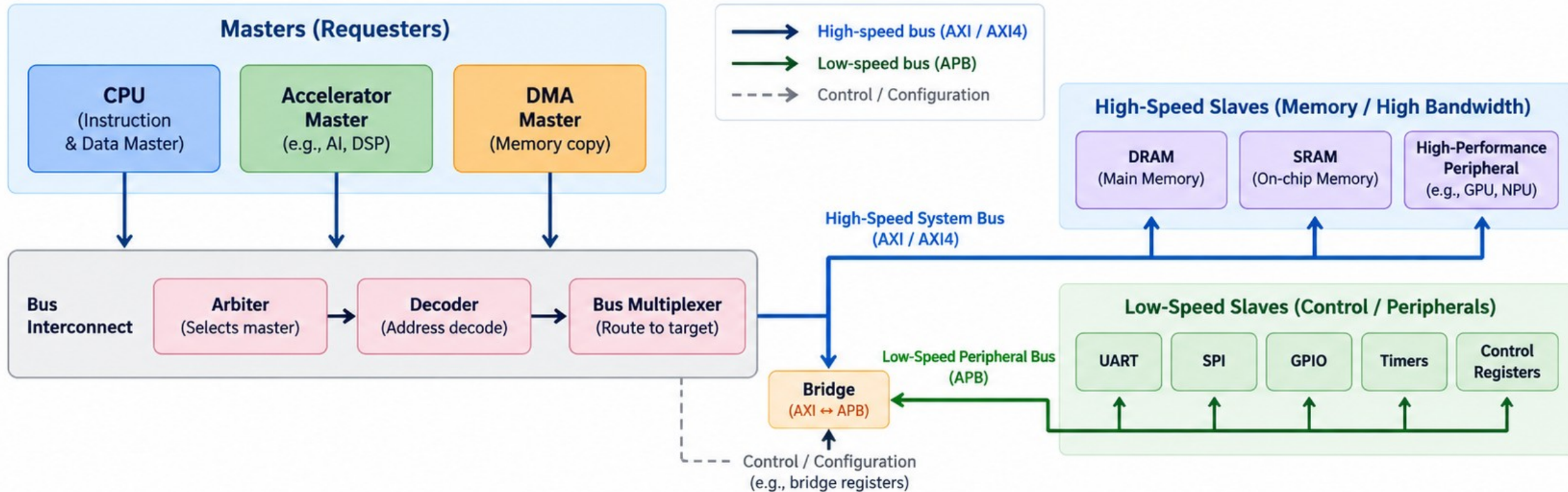
#	Protocol	Key Characteristics	Typical Use Cases	Advantages	When to Choose
01	 CSE Fabric (Custom / Domain-specific)	- Very small, low-bandwidth control path - Simple and lightweight - Often used for control/status	- Core control signals - Configuration and status - Low-rate management traffic	- Minimal area and power - Simple to implement - Low latency	When you need a small, low-bandwidth control interconnect.
02	 APB-like bus (AMBA APB)	- Simple peripheral registers - Low bandwidth, non-pipelined - Easy to integrate	- Control and status registers - Simple peripherals (UART, GPIO) - Low-speed devices	- Very simple design - Low area and power - Widely supported	When connecting simple, low-speed peripherals.
03	 Wishbone (Open Standard)	- Open and compact - Flexible and easy to integrate - Supports pipelining (optional)	- FPGA-based systems - Open-source SoC designs - Moderate bandwidth peripherals	- Open standard (no licensing) - Simple and flexible - Good FPGA ecosystems	When you need a simple, open interconnect for FPGA or custom designs.
04	 AXI4 (AMBA AXI4)	- High bandwidth, pipelined - Multiple outstanding transfers - Separate read/write channels - Supports bursts and IDs	- High-performance masters/slaves - CPUs, DMA, accelerators - On-chip and off-chip memory access	- High throughput - Scalable and flexible - Widely adopted ecosystem - Supports complex systems	When you need high bandwidth and advanced features (e.g., multiprocessing, accelerators).
05	 AXI Stream (AMBA AXIS)	- Address-free, continuous data - Valid/ready handshake - Optimized for streaming	- Video, image and sensor data - DSP and AI/ML pipelines - High-throughput data streaming	- Low latency - Simple streaming interface - Ideal for high-throughput data	When you need to move continuous data streams without addresses.
06	 TileLink (RISC-V Ecosystem)	- Coherent, generator-based - Supports cache coherence - Flexible, scalable topology - Used in modern RISC-V SoCs	- RISC-V multi-core systems - Cache-coherent interconnect - Generated with tools (e.g., Chipyard) - On-chip subsystem communication	- Coherence support - Highly configurable - Strong RISC-V ecosystem - Scales from simple to complex SoCs	When building RISC-V systems that need coherence and scalable, generator-based interconnect.

Right protocol, right system — match the interconnect to your performance, complexity and ecosystem needs.

Bus Hierarchy Keeps Fast and Slow Domains Manageable

REAL SOC_s ARE NOT ONE FLAT BUS

Bridges isolate protocol, clock and performance differences across the chip.



Isolation

- Keep CPU, DMA, accelerators and memory at high bandwidth
- Separate fast and slow domains



Peripheral Domain

- Serve UART, timers, GPIO and low-rate control registers
- Simpler, lower power, different clock



Bridge + CDC

- Translate protocols (e.g., AXI to APB)
- Handle clock domain crossings
- Isolate timing and maintain system stability

A hierarchical bus architecture enables high performance, low power and a scalable, manageable SoC.

Build the SoC Address Map Before Writing the Driver

Every memory and MMIO region needs a non-overlapping address range, access policy and owner.

SoC Address Map (Example)

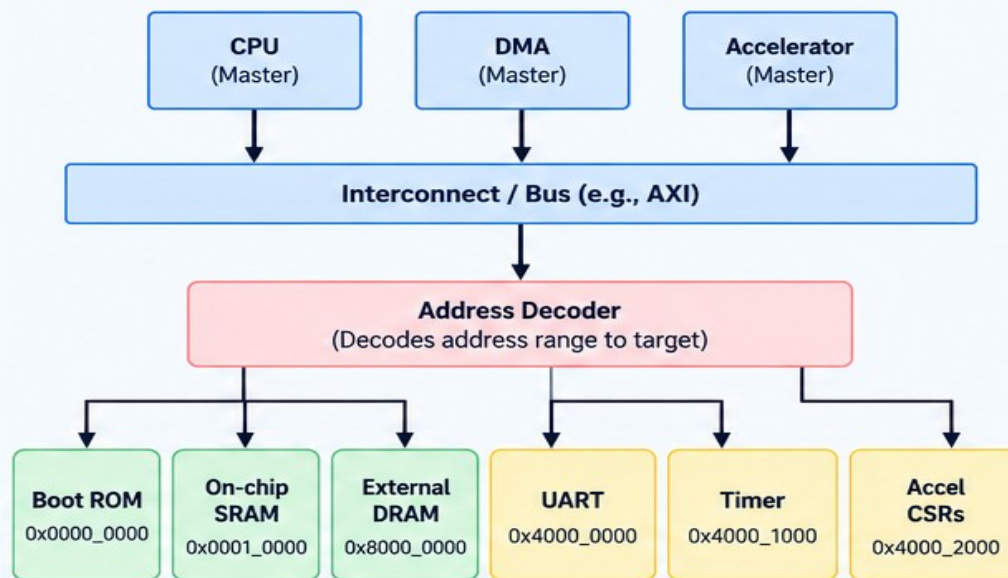
Address Range	Size	Region / Device	Type	Access
0x0000_0000 – 0x0000_7FFF	32 KB	Boot ROM	Memory	R-X
0x0001_0000 – 0x0001_FFFF	1 MB	On-chip SRAM	Memory	R/W/X
0x8000_0000 – 0x8FFF_FFFF	256 MB	External DRAM	Memory	R/W/X
0x4000_0000 – 0x4000_0FFF	4 KB	UART	MMIO	R/W
0x4000_1000 – 0x4000_1FFF	4 KB	Timer	MMIO	R/W
0x4000_2000 – 0x4000_2FFF	4 KB	Accelerator (CSRs)	MMIO	R/W
0x5000_0000 – 0x5000_FFFF	1 MB	Debug & System Control	MMIO	R/W

Boot / Code region

Main memory

Peripheral MMIO

SoC Address Decoding and Interconnect



1. Decode



- Interconnect routes each address to one target based on address range.
- No overlapping ranges.

2. Protection



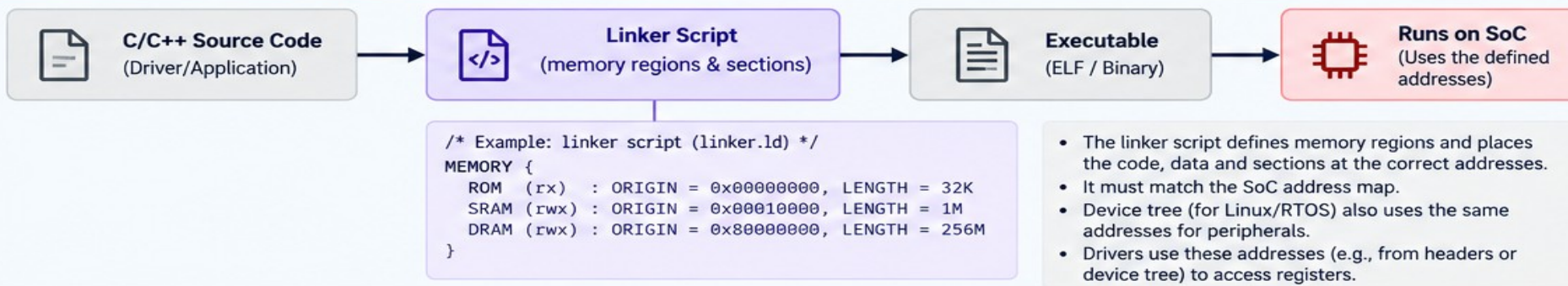
- Set access permissions (e.g., R/W/X).
- Restrict master access (e.g., CPU, DMA, accelerators).
- Prevent illegal or unintended accesses.

3. Software Contract



- Linker, device tree and drivers must use the same address map.
- Addresses are the software–hardware interface.
- Change the map → update linker script, device tree, drivers.

Where the Address Map Lives in the Software Flow



Define the address map early — it connects hardware, the linker, the OS and the drivers.

Synchronize Hardware and Software Explicitly

COMPLETION MUST BE UNAMBIGUOUS

The system needs defined ownership, ordering and timeout behavior.



Polling

Simple software repeatedly reads status; useful for short operations.



Interrupt

Hardware notifies the CPU; useful for long or asynchronous work.



Ordering

Fences, ready / valid rules and ownership prevent stale or premature access.

Agenda

The lecture follows the same sequence used in a real co-design project.

Co-design foundations

System architecture methodology

Accelerator architecture and RISC-V integration

SoC, memory and system software

Performance, area and power trade-offs

Implementation frameworks and final workflow

Place Data Where the Accelerator Can Reuse It Efficiently

01	Registers	Immediate operands and partial results
02	FIFO	Decouple producer and consumer timing
03	Local SRAM	Software- or hardware-managed working set
04	Shared cache	Reuse with processor-visible coherence rules
05	DRAM	Large capacity with higher latency
06	Storage / I/O	External source or destination

Scratchpads Trade Automation for Predictability

Local storage policy determines who manages placement and when data movement occurs.

Scratchpad

- Directly addressed local SRAM
- Predictable access latency
- Explicit DMA or software management
- Good for known tiled working sets

Cache

- Automatic placement and replacement
- Transparent to most software
- Variable hit and miss behavior
- May require coherence with other masters

A DMA Transfer Has Three Ownership Phases

1 · Configure

Software owns buffers

Allocate and prepare memory, write descriptors, then release work to the accelerator.

2 · Transfer

Hardware owns in-flight work

Read inputs, process data and write results while respecting bus and cache rules.

3 · Complete

Software regains ownership

Observe done or interrupt, synchronize memory and safely consume the results.

Non-Coherent DMA Can Produce Stale Data

CPU caches and external masters may hold different views of the same buffer.

Failure scenario

CPU modifies a cached input buffer

DMA reads older DRAM contents

Accelerator writes a result to DRAM

CPU reads an older cached result

Before DMA read

Clean or flush CPU-modified cache lines.

After DMA write

Invalidate stale CPU cache lines before reading.

Alternative

Use coherent interconnect support where available and justified.

Bandwidth, Bursts and Buffers Must Be Sized Together

KEEP THE COMPUTE PIPELINE FED

Peak arithmetic throughput is useless when the data path cannot sustain it.



Bandwidth

Required bytes per second must fit the bus and memory system.



Bursts

Long aligned transactions reduce address and arbitration overhead.



Buffers

Absorb latency variation and separate memory timing from compute timing.

Multiple Masters Require Arbitration and QoS

CPU, DMA and accelerators compete for shared memory and interconnect resources.

Questions to answer

Who receives priority?

How long can another master be blocked?

Are bursts limited or preemptible?

Which traffic has a deadline?

How is fairness measured?

Latency-sensitive

Control and real-time traffic may need bounded delay.

Bandwidth-sensitive

Bulk DMA needs sustained service to remain efficient.

System policy

QoS should reflect workload importance, not just hardware proximity.

Hardware Becomes Useful Through a Complete Software Path

01	Application	Requests a user-visible operation
02	Library	Presents a stable functional API
03	Runtime	Schedules work and manages buffers
04	Driver	Programs registers, DMA and interrupts
05	Firmware / OS	Provides memory, protection and device discovery
06	Accelerator	Executes and reports status

Connect Software to the Accelerator at the Right Level

PROGRAMMABILITY IS A DESIGN OUTPUT

The chosen interface should hide hardware detail without hiding essential control.

API

Application-facing function with clear inputs, outputs and errors.

Driver

Register access, memory mapping, interrupt handling and recovery.

Compiler

Inline assembly, intrinsics, custom operations or code generation.

Runtime

Queues, dependencies, buffer reuse and multi-accelerator scheduling.

Agenda

The lecture follows the same sequence used in a real co-design project.

Co-design foundations

System architecture methodology

Accelerator architecture and RISC-V integration

SoC, memory and system software

Performance, area and power trade-offs

Implementation frameworks and final workflow

Verification Progresses from Behavior to Deployment Evidence

1 · Model

2 · RTL

3 · Prototype

Functional + QEMU

Verify results, register semantics, memory map and software sequence early.

Simulation + assertions

Verify protocol, corner cases, backpressure, interrupts and cycle behavior.

FPGA + benchmarks

Measure timing, resources, end-to-end performance and software stability.

Measure More Than Raw Kernel Speedup

SYSTEM METRICS DRIVE THE DECISION

A balanced scorecard prevents a locally fast design from becoming a system-level regression.

Performance

Latency, throughput, speedup and utilization.

Efficiency

Energy per task and performance per watt.

Cost

Area, FPGA resources, memory and bandwidth.

Overhead

CPU time, setup, transfers, synchronization and software complexity.

$$\text{Speedup} = 1 / ((1 - P) + P / S)$$

Find the real fraction

P = ACCELERATED SHARE

Measure how much of the complete application can actually use the accelerator.

Design implication

Accelerate a meaningful fraction and reduce communication—not only arithmetic latency.

S = kernel speedup

Hardware may dramatically improve the selected kernel while leaving other work unchanged.

System limit

Setup, transfers, synchronization and serial code cap end-to-end improvement.

Latency and Throughput Answer Different Questions

A design optimized for the first result may differ from one optimized for continuous work.

Latency

- Time from request to completed result
- Critical for control and dependent tasks
- Affected by setup and synchronization
- Often improved by tight coupling

Throughput

- Results or data processed per unit time
- Critical for streams and large batches
- Affected by pipeline rate and bandwidth
- Often improved by buffering and concurrency

Hardware Parallelism Consumes Physical Resources

MORE LANES ARE NOT FREE

Resource growth can reduce achievable frequency, increase routing pressure or exceed the target device.

Logic

Control, routing, comparisons and state consume LUTs or gates.

Arithmetic

Multipliers and MACs consume DSPs or custom datapaths.

Storage

Registers, FIFOs and buffers consume flip-flops, SRAM or BRAM.

Interconnect

Wider buses and more ports increase routing and arbitration complexity.

Performance and Power Must Be Evaluated Together

ENERGY IS WORK; POWER IS RATE

A faster architecture can finish sooner yet still violate peak power or thermal limits.

Dynamic power

Switching activity, capacitance, voltage and frequency.

Static power

Leakage and always-on resources.

Energy per task

Total energy required for one complete workload.

Power controls

Clock gating, power states, voltage and activity management.

The Bottleneck Can Move from Compute to Memory

Increasing arithmetic parallelism helps only while data arrives quickly enough.

Compute-bound

Functional units remain busy

More lanes or deeper pipelines may help

Reuse reduces repeated work

Operator efficiency dominates

Memory-bound

Units wait for operands

More lanes increase contention

Tiling, buffering or compression may help

Bandwidth and locality dominate

Explore Parameters Systematically Instead of Guessing

- | | | |
|----|--------------------------|---|
| 01 | Select parameters | Lanes, precision, buffer, bus width and frequency |
| 02 | Define workloads | Representative sizes and operating conditions |
| 03 | Automate builds | Generate consistent design variants |
| 04 | Measure | Performance, resources, power and correctness |
| 05 | Compare | Normalize metrics and expose bottlenecks |
| 06 | Refine | Focus the next sweep on promising regions |

Agenda

The lecture follows the same sequence used in a real co-design project.

Co-design foundations

System architecture methodology

Accelerator architecture and RISC-V integration

SoC, memory and system software

Performance, area and power trade-offs

Implementation frameworks and final workflow

Frameworks Operate at Different Abstraction Levels

CHOOSE THE RIGHT LEVER

A project may combine several tools rather than forcing one framework to cover the entire flow.

HLS

Generate datapaths from constrained high-level descriptions.

Chisel + Chipyard

Build parameterized RISC-V cores, tiles and tightly coupled accelerators.

LiteX + Migen

Compose Python-configured FPGA SoCs, memories and peripherals.

FuseSoC

Package, reuse and build HDL IP across tools and targets.

Increase Cost and Fidelity Only as Evidence Improves

01	CPU baseline	Compile, run, profile and preserve reference results
02	Functional model	Validate algorithm, precision and corner cases
03	QEMU / transaction model	Validate address map, driver and synchronization
04	RTL / generator	Implement interfaces, control and datapath
05	FPGA prototype	Measure timing, resources and complete workloads
06	ASIC path	Synthesize, verify and close physical constraints

Final Design Checklist and Exercise

A defensible co-design links every hardware choice back to a workload requirement and measured system effect.

Before accepting a design

Problem and baseline are documented

Partitioning is justified

Control and data interfaces are explicit

Memory, coherence and software are complete

Performance, area and power are measured

Verification covers the full operating flow

Exercise

Select one application hotspot and propose an accelerator.

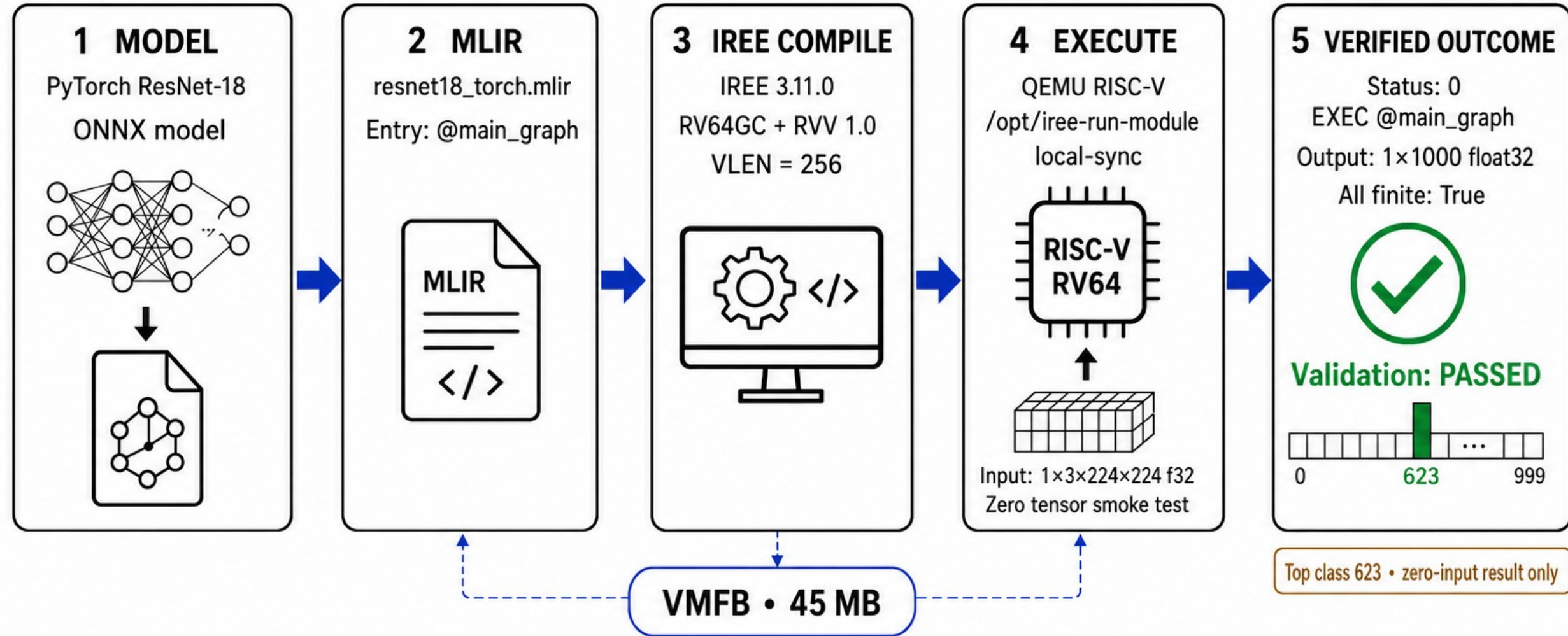
Architecture

Choose integration, memory and software interfaces.

Defence

Explain the expected benefit, cost, risks and validation plan.

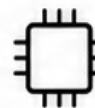
ResNet-18 MLIR → RISC-V RVV256 Execution Pipeline



Compiler: /usr/local/bin/iree-compile



Runtime: /opt/iree-run-module



QEMU: /opt/riscv/bin/qemu-riscv64



Workspace: /shared/school26/resnet18_iree

ChipCluster ResNet-18 on RISC-V RVV256 — Execution Report

1. Problem Identified

- Original project: /shared/projects/resnet18_iree (read-only for school26)
- Initial runtime path \$HOME/iree-build-riscv did not exist
- Shared build runtime rejected the VMFB: required features [Ch]
- Therefore compilation existed, but inference output was not produced

2. Correct Configuration

WORKDIR	/shared/school26/resnet18_iree
COMPILER	/usr/local/bin/iree-compile
RUNTIME	/opt/iree-run-module
QEMU	/opt/riscv/bin/qemu-riscv64
SYSROOT	/opt/iree-riscv-toolchain/sysroot
CPU	RV64 + RVV 1.0, VLEN=256, ELEN=64

3. Commands to Reproduce

Panel A — Setup

```
cd /shared/school26/resnet18_iree
mkdir -p input output logs vmfb

RUNNER=/opt/iree-run-module
QEMU=/opt/riscv/bin/qemu-riscv64
SYSROOT=/opt/iree-riscv-toolchain/sysroot
```

Panel B — Compile

```
iree-compile mlir/resnet18_torch.mlir \
  --iree-hal-target-device=local \
  --iree-hal-local-target-device-backends=llvm-cpu \
  --iree-llvmcpu-target-triple=riscv64 \
  --iree-llvmcpu-target-cpu=generic-rv64 \
  --iree-llvmcpu-target-abi=lp64d \
  --iree-llvmcpu-target-cpu-features="+m,+a,+f,+d,+c,+v,+zv1256b" \
  --riscv-v-fixed-length-vector-lmul-max=8 \
  --iree-opt-level=O2 \
  -o vmfb/ResNet18_riscv64_rvv256_v1p0.vmfb
```

Panel C — Execute and Validate

```
$QEMU -cpu rv64,zve64d=true,vlen=256,elen=64,vext_spec=v1.0 \
  -L $SYSROOT $RUNNER --device=local-sync \
  --module=vmfb/ResNet18_riscv64_rvv256_v1p0.vmfb \
  --function=main_graph --input="1x3x224x224xf32=0" \
  --output=@output/resnet18_rvv256_opt_output.npy

python3 -c "import numpy as np; x=np.load('output/resnet18_rvv256_opt_output.npy'); print(x.shape,x.dtype,np.isfinite(x).all())"
```

Writable Project

Torch MLIR

IREE Compile

RVV256 VMFB

QEMU + IREE Runtime

NumPy Output

Validation Passed

4. Verified Results

- | | |
|--|--|
| <ul style="list-style-type: none">• QEMU/IREE status: 0• Runtime message: EXEC @main_graph• VMFB: 45 MB• Output: 4.1 KB, shape (1, 1000), float32 | <ul style="list-style-type: none">• All values finite: True• Top-5 indices: [623, 111, 499, 813, 596]• RVV256 ResNet-18 validation: PASSED |
|--|--|

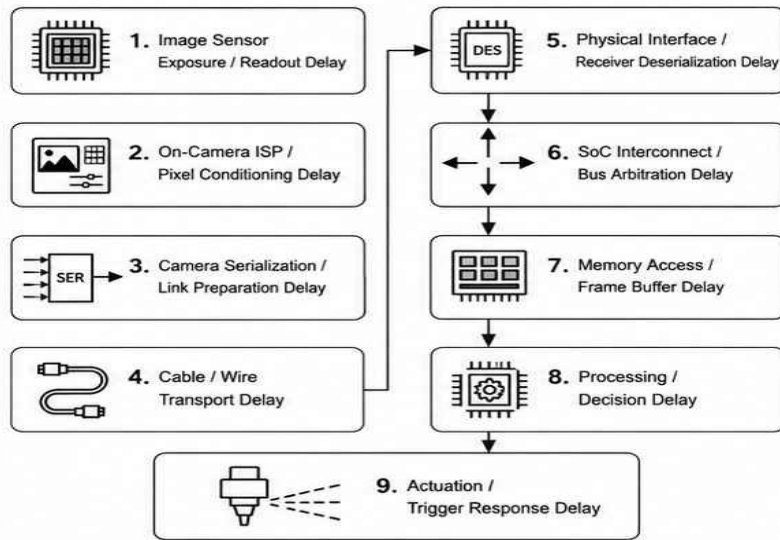
Caution note:

All-zero input validates the pipeline only;
class 623 is not a meaningful image prediction.

A Multi-Interface Real-Time Vision Architecture for Rice Sorting: CPU/GPU Benchmarking and Deployed SoC–FPGA Evaluation

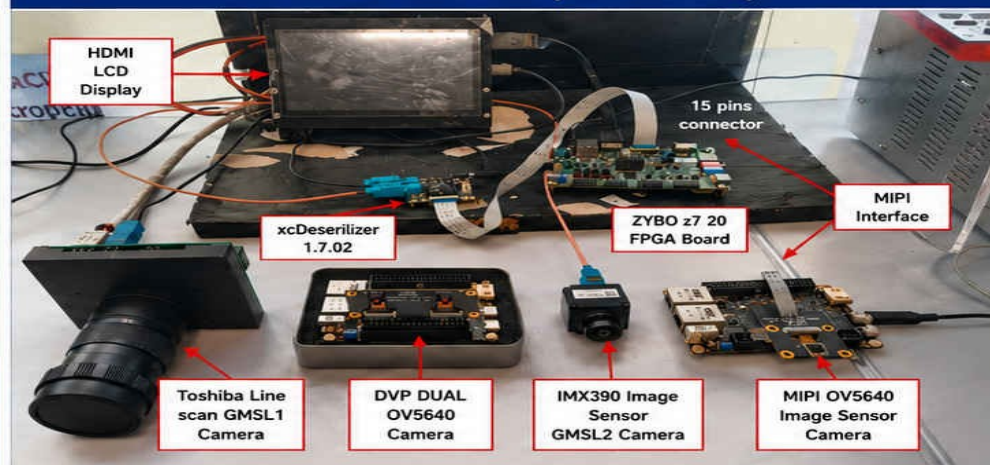
TASSADAQ HUSSAIN^{1,2}, SOLTAN ABED ALHARBI³, MIR ASMA⁴, and ALI SALEH ALSHOMRANI^{5,*},

1) REAL-TIME VISION PIPELINE FOR CLOSED-LOOP RICE SORTING



Deterministic end-to-end latency for timely pneumatic ejection.

2) TARGETED PROCESSING PLATFORMS AND INTERFACES (EVALUATED)



CPU Platform (Baseline)
Intel NUC7i5DN i5-7300U



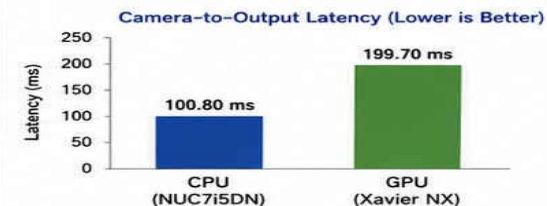
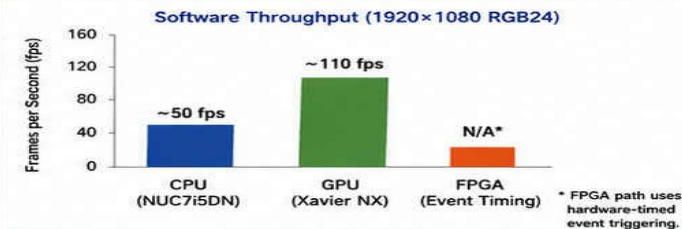
GPU Platform (Embedded)
NVIDIA Jetson
Xavier NX 16 GB



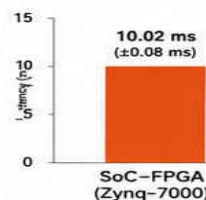
SoC–FPGA Platform
Xilinx Zynq-7000
XC7Z020



3) KEY RESULTS (EXPERIMENTAL)



FPGA Event-to-Trigger Latency (Deterministic Path)

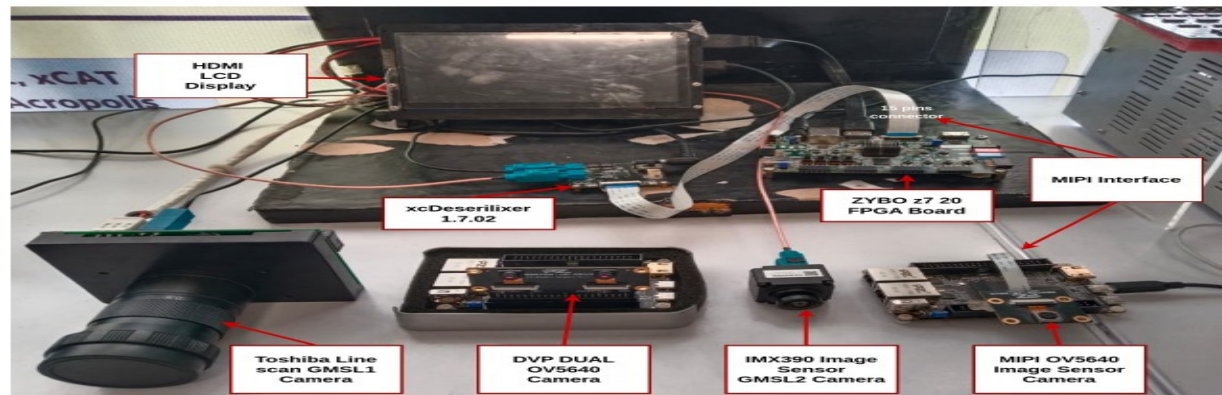


Classification Performance (2,000,000 observations × 5 trials)





FIGURE 6: Developed and fully integrated rice colour-sorting prototype: (a) front view showing the hopper, chute assembly, operator panel, HMI, product outlets, and FPGA development setup; (b) close view of the operator interface, processing hardware, and output section; (c) rear view showing the hopper, feeding mechanism, illuminated inspection chamber, and supporting structure; and (d) side view showing the inclined material-flow path, inspection enclosure, pneumatic lines, and ejector assembly.

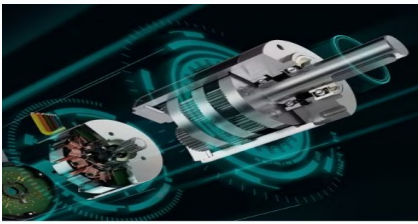


(a) Multi-interface camera and XC7Z020 validation bench, including line-scan, DVP, and MIPI paths.



(b) Jetson Xavier NX 16 GB six-channel GMSL2/GMSL1 camera bench.

FIGURE 7: Experimental hardware used for the CPU, embedded-GPU, camera-interface, and SoC-FPGA evaluation.



AN ENERGY EFFICIENT & LOW-COST RISC-V BASED BLDC MOTOR CONTROLLER

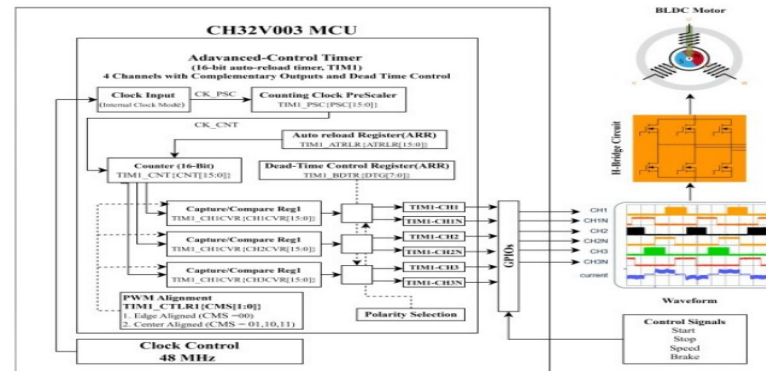
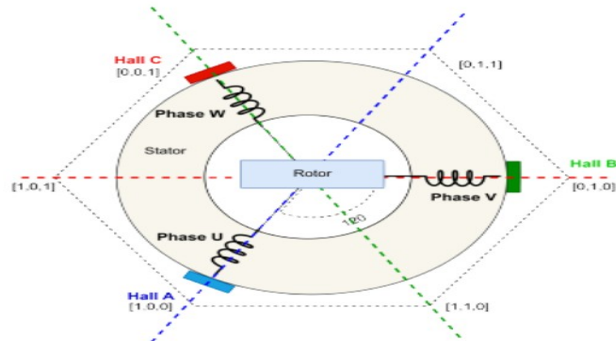
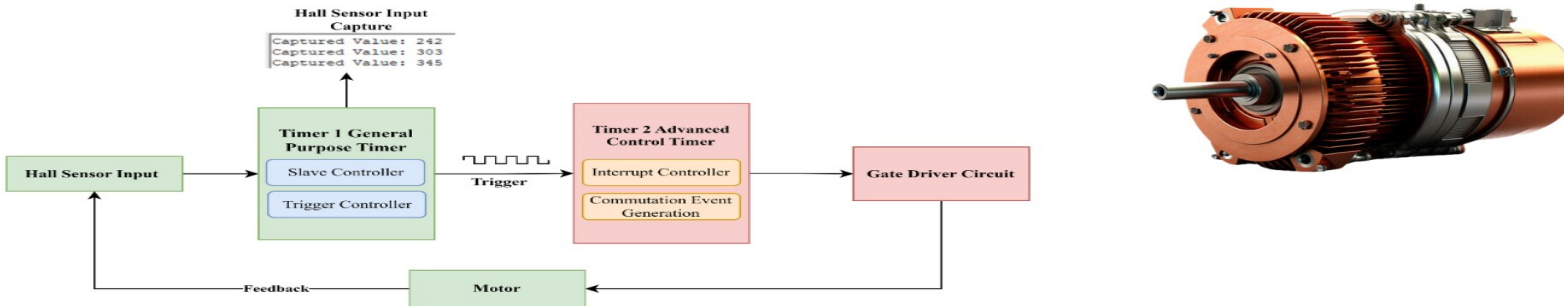


IMPORTANCE

BLDC motors are widely used for their high efficiency reliability and lesser maintenance issues, but existing controllers struggle to provide balance in power consumption, control features and cost-effectiveness. This study develops an energy-efficient BLDC motor controller using a RISC-V processor with low-level control speed and position feedback, optimizing power usage while maintaining precise motor control.

SALIENT FEATURES

- Utilizes a low-cost 48MHz RISC-V processor for improved energy efficiency, affordability and advanced motor control.
- Optimized power consumption while maintaining precise speed and torque control.
- Implements low level programmed control algorithms for better performance.
- Evaluated based on power efficiency, speed control accuracy, and torque response.
- Outperforms existing programmed motor controllers, ensuring reliable and efficient motor operation.



POST-LAYOUT ASIC CORE

RISC-V SoC • RTL-to-GDSII



TECHNOLOGY

IHP SG13G2

130 nm CMOS



LOGIC + MEMORY

3,881 standard cells

1 × SRAM macro

1024 × 32 bits

SRAM counted separately

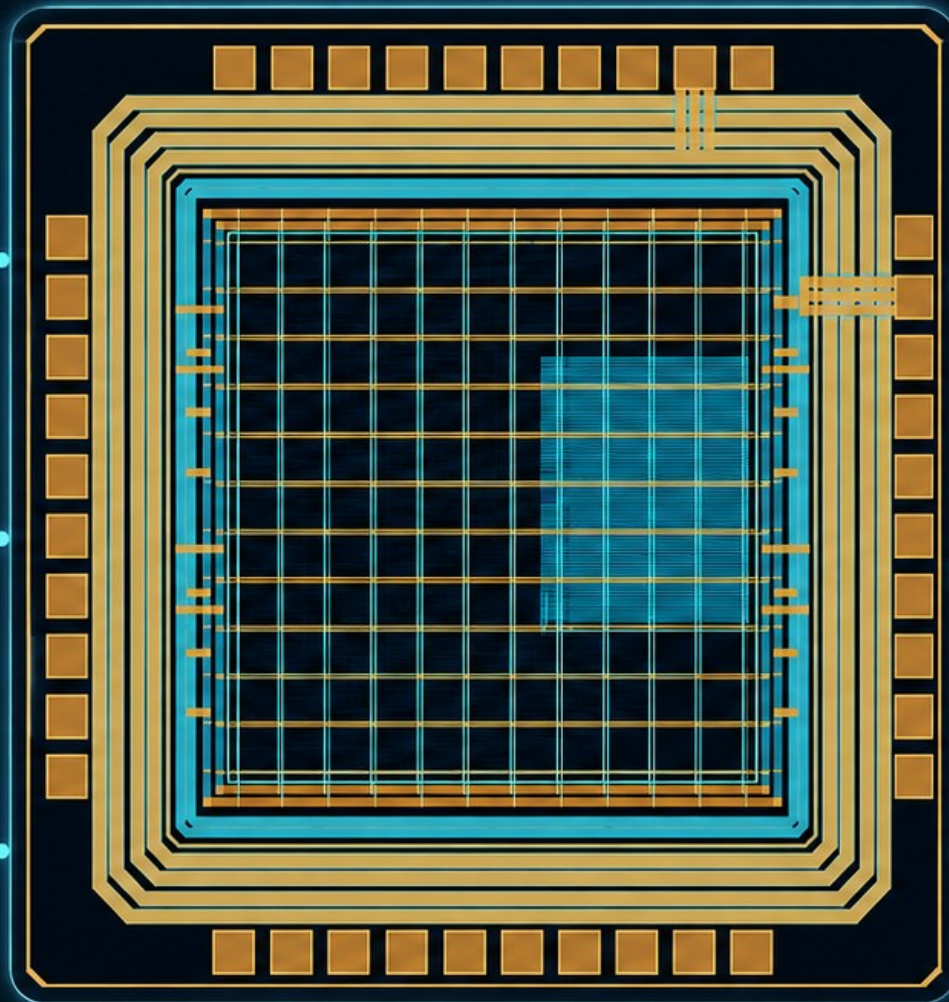


POWER

3.43 mW

Estimated post-layout

Switching: 3.32 mW



DIE + CORE AREA

Die: 1.60 × 1.60 mm

Core: 0.752 mm²



CORE UTILIZATION

26.32%

Std-cell region: 9.45%



CLOCK / TIMING

Closed at 3 PVT corners

Setup margin: +1.11 ns

Hold margin: +0.125 ns

Clock rate not reported



0

SETUP/HOLD
VIOLATIONS



0

ROUTE DRC



0

ANTENNA
VIOLATIONS



WORST IR DROP

0.653 mV



Remaining checks: 13 slew • 32 fanout • 2 density

CAID SoC — What We Built & How the Project Is Organized

The Project

- A complete SoC built around a custom RISC-V (RV32I) core
- Core + memory + GPIO connected over a Wishbone bus
- GCC-compatible — runs real, compiled C programs
- Taken from RTL design through simulation, real FPGA hardware, and a full ASIC layout

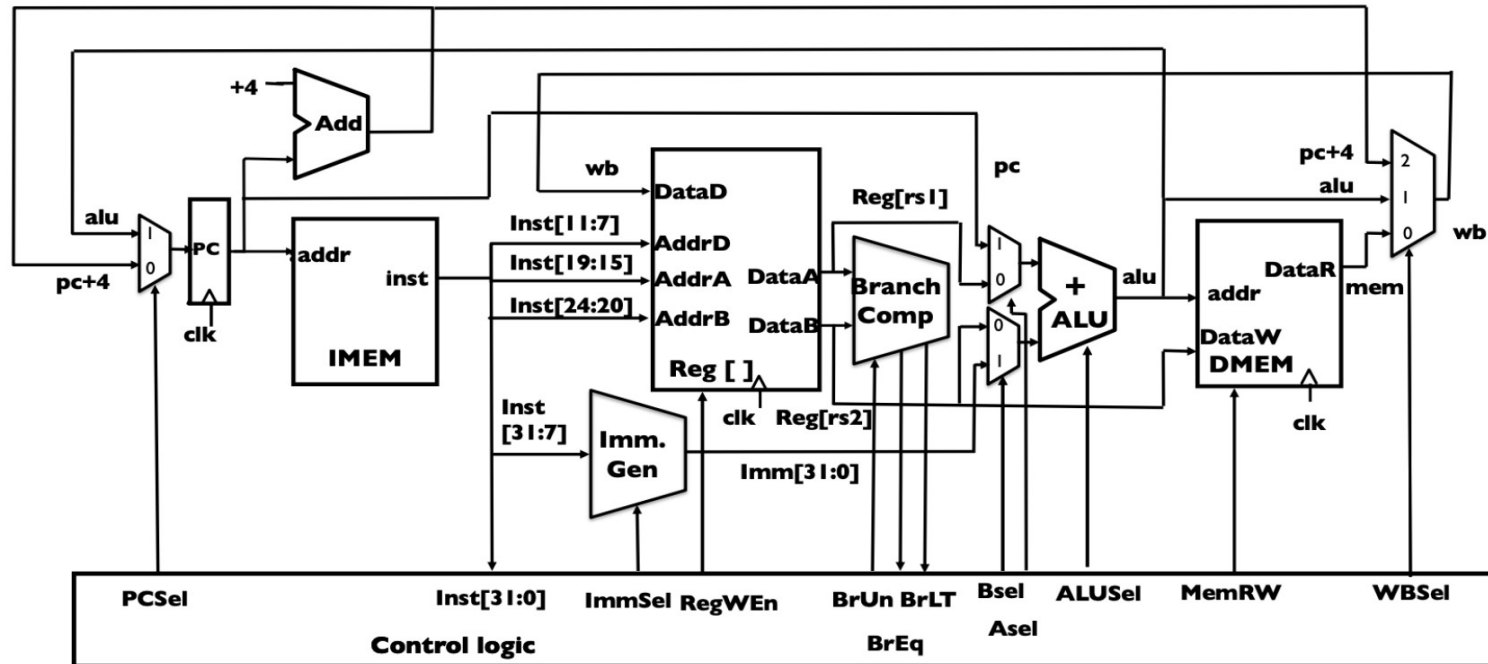
This Project Is Divided six phases



Core Design — Single-Cycle RV32I RISC-V Core

What We Designed

- A single-cycle RISC-V core, designed from scratch in Verilog
- Full Datapath: PC, ALU, Register File, Immediate Generator, Branch Comparator, Control Unit

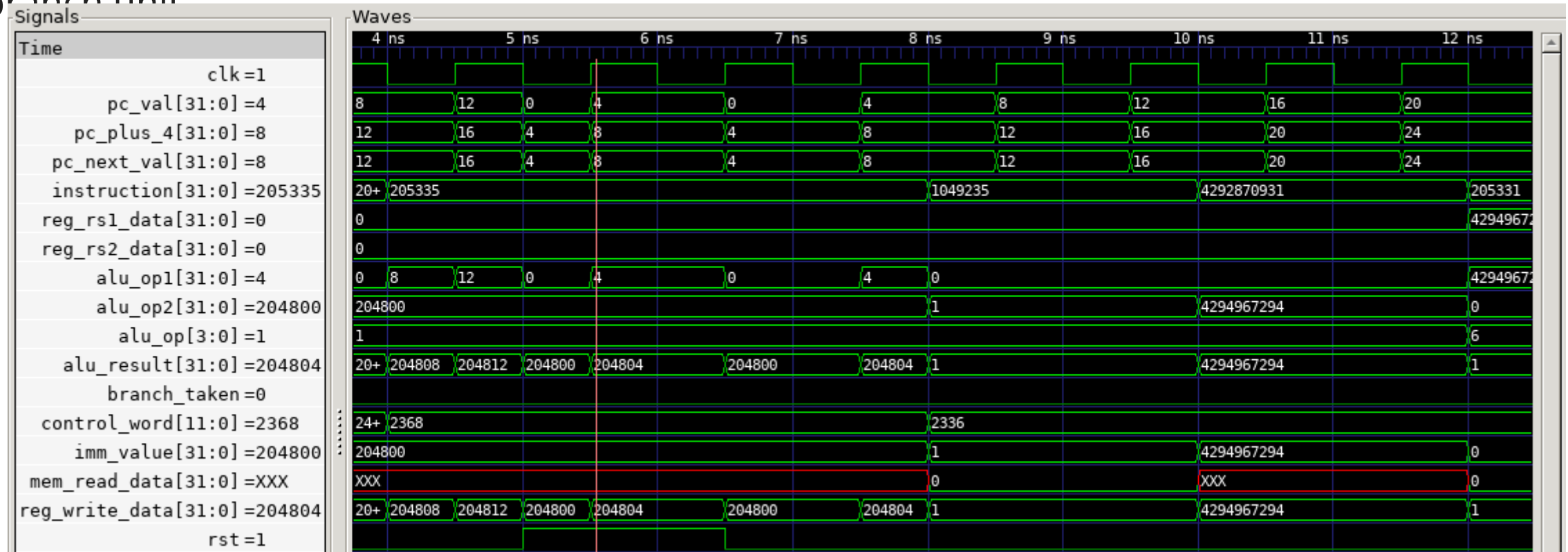


<https://media.cheggcdn.com/>

Core Verification & Validation

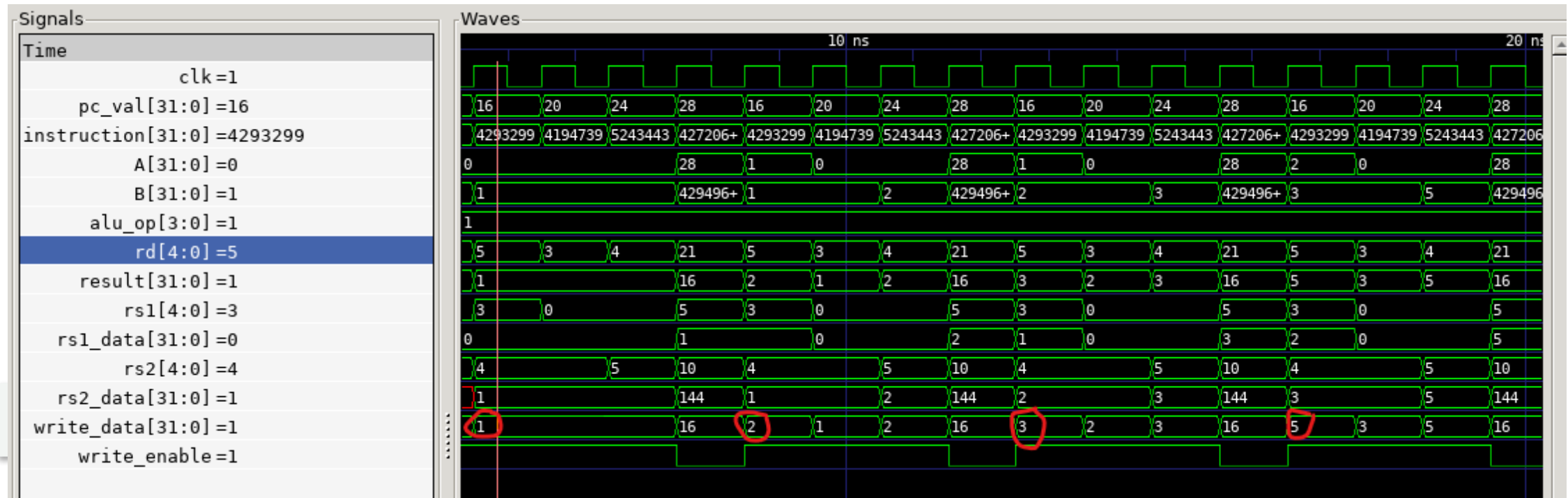
Full RV32I instruction set — R, I, S, B, U & J-type

Functional units individually: ALU, register file, immediate generator, branch unit



Software Ecosystem Enablement — GCC Compatibility

C Source → RISC-V GCC → ELF Binary → .mem Hex File → Instruction Memory → RISC-V Core Execution → GTKWave Verification → Fibonacci Result (1, 2, 3, 5, 8 ... 144)

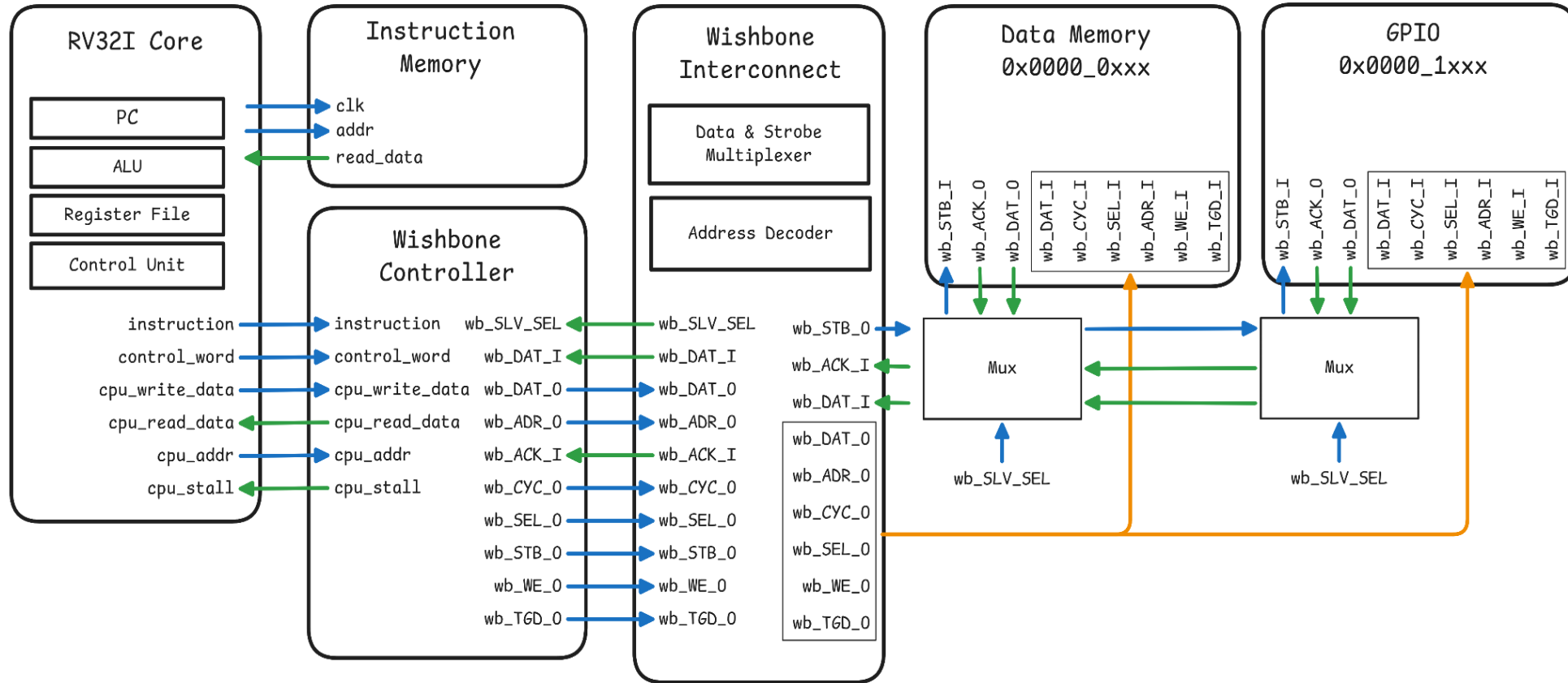


RISC-V Core Development

Core Verification &
Validation

Software Ecosystem
Enablement

SoC & Wishbone Integration



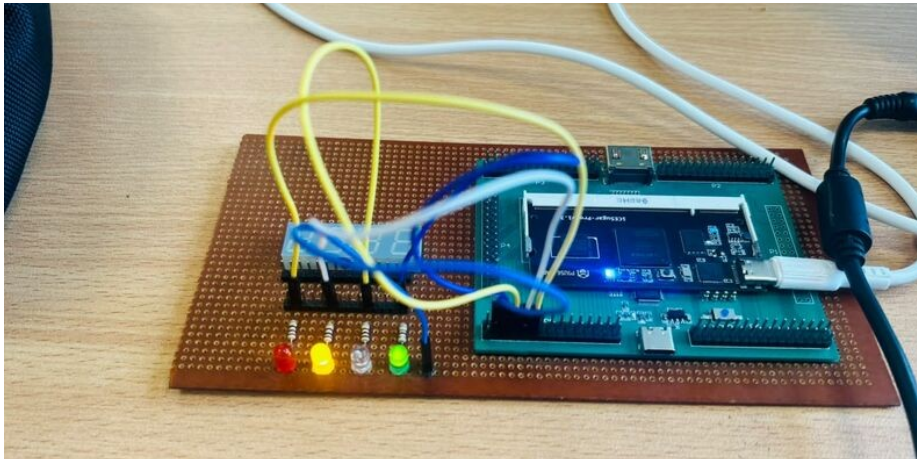
Multi-FPGA Deployment

RTL → Synthesis → Place & Route → Bitstream → Hardware Testing

- **Synthesis:** Yosys
- **Place & Route:** nextpnr
- **Programming:** openFPGALoader

ECP5 Colorlight i9

- **FPGA:** Lattice ECP5 LFE5U-45F (CABGA381)
- **Onboard Clock:** 25 MHz
- **Timing-Closure Target:** 65 MHz
- **LUT Utilization:** 1193 / 43848 (2%)



```
Info: Device utilisation:
Info:      TRELLIS_IO:      8/    245    3%
Info:      DCCA:            1/     56    1%
Info:      DP16KD:           0/    108    0%
Info:      MULT18X18D:       0/     72    0%
Info:      ALU54B:           0/     36    0%
Info:      EHXPPLL:          0/      4    0%
Info:      EXTREFB:          0/      2    0%
Info:      DCUA:             0/      2    0%
Info:      PCSCLKDIV:        0/      2    0%
Info:      IOLOGIC:          0/    160    0%
Info:      SIOLOGIC:         0/     85    0%
Info:      GSR:              0/      1    0%
Info:      JTAGG:             0/      1    0%
Info:      OSCG:             0/      1    0%
Info:      SEDGA:            0/      1    0%
Info:      DTR:              0/      1    0%
Info:      USRMCLK:          0/      1    0%
Info:      CLKDIVF:          0/      4    0%
Info:      ECLKSYNCB:        0/     10    0%
Info:      DLLDELD:          0/      8    0%
Info:      DDRDLL:           0/      4    0%
Info:      DQSBUFM:          0/     10    0%
Info:      TRELLIS_ECLKBUF:   0/      8    0%
Info:      ECLKBRIDGECS:     0/      2    0%
Info:      DCSC:             0/      2    0%
Info:      TRELLIS_FF:       44/  43848    0%
Info:      TRELLIS_COMB:     1193/  43848    2%
Info:      TRELLIS_RAMW:     32/   5481    0%
```

RISC-V Core
Development

Core Verification &
Validation

Software Ecosystem
Enablement

SoC & Wishbone
Integration

Multi-FPGA
Deployment

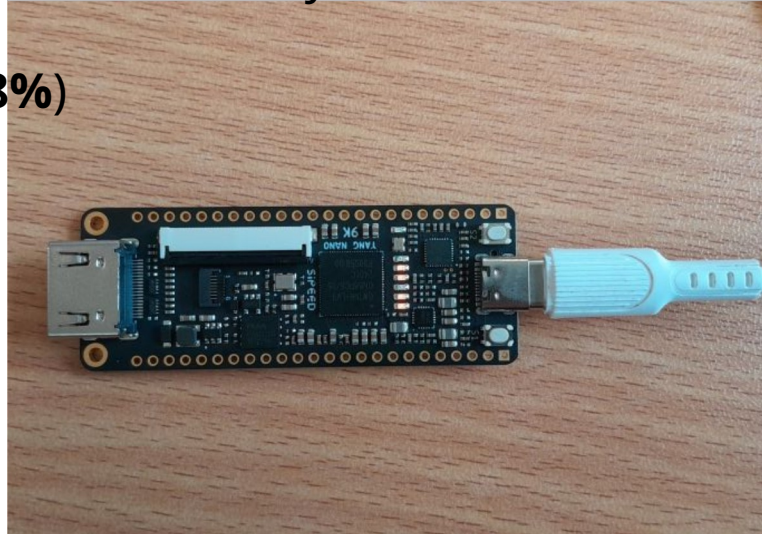
Multi-FPGA Deployment

RTL → Synthesis → Place & Route → Bitstream → Hardware Testing

- **Synthesis:** Yosys
- **Place & Route:** nextpnr
- **Programming:** openFPGALoader

Sipeed Tang Nano 9K

- **FPGA:** Gowin GW1N-9C
- **Onboard Clock:** 27 MHz
- **PLL:** Not used; oscillator drives the SoC directly
- **Post-P&R Fmax:** 32.57 MHz
- **LUT Utilization:** 1611 / 8,640 (18%)



```
Info: Device utilisation:
Info:          VCC:      1/      1 100%
Info:          IOB:      7/     274   2%
Info:          LUT4:    1611/    8640  18%
Info:          OSER16:    0/      80   0%
Info:          IDES16:    0/      80   0%
Info:          IOLOGICI:  0/     276   0%
Info:          IOLOGICO:  0/     276   0%
Info:          MUX2_LUT5:  590/    4320  13%
Info:          MUX2_LUT6:  280/    2160  12%
Info:          MUX2_LUT7:  123/    1080  11%
Info:          MUX2_LUT8:   52/    1080   4%
Info:          ALU:      138/    6480   2%
Info:          GND:       1/       1 100%
Info:          DFF:      43/    6480   0%
Info:          RAM16SDP4:  32/     270  11%
Info:          BSRAM:     0/      26   0%
Info:          ALU54D:     0/      10   0%
Info:          MULTADDALU18X18: 0/      10   0%
Info:          MULTALU18X18:    0/      10   0%
Info:          MULTALU36X18:    0/      10   0%
Info:          MULT36X36:     0/       5   0%
Info:          MULT18X18:     0/      20   0%
Info:          MULT9X9:      0/      40   0%
Info:          PADD18:      0/      20   0%
Info:          PADD9:       0/      40   0%
Info:          GSR:        1/       1 100%
Info:          OSC:        0/       1   0%
Info:          rPLL:       0/       2   0%
Info:          FLASH608K:    0/       1   0%
Info:          BUFG:       0/      22   0%
Info:          DQCE:       0/      24   0%
Info:          DCS:        0/       8   0%
Info:          DHCEN:      0/      24   0%
Info:          CLKDIV:     0/       8   0%
Info:          CLKDIV2:    0/      16   0%
```

RISC-V Core
Development

Core Verification &
Validation

Software Ecosystem
Enablement

SoC & Wishbone
Integration

Multi-FPGA
Deployment

ASIC & Silicon Readiness — GDS Flow

Design Flow

- LibreLane — open-source RTL-to-GDSII flow

PDK & Library

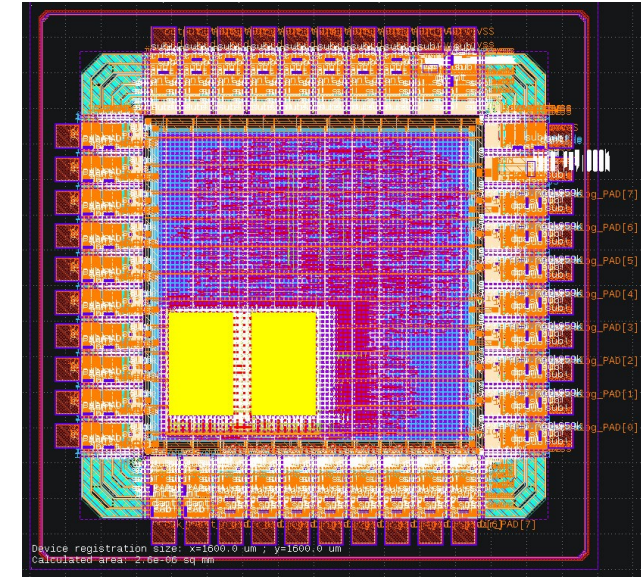
- IHP SG13G2 — 130nm open-source PDK
- sg13g2 standard-cell library + SRAM macro (data memory)

Key Specifications

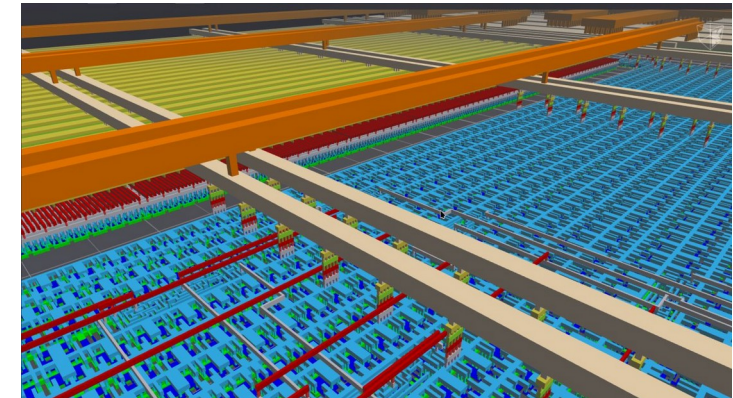
- Clock Target: 50 MHz (86MHz @typ corner)
- Die area: 1.6 mm x 1.6 mm
- ~3969 standard cells placed & routed

Issues

- Slew & Cap Violations
- ~1600 DRC errors
- No programming support



Full-chip layout — 1.6 x 1.6 mm die



3D routing detail view

RISC-V Core
Development

Core Verification &
Validation

Software Ecosystem
Enablement

SoC & Wishbone
Integration

Multi-FPGA
Deployment

ASIC & Silicon
Readiness

POST-LAYOUT ASIC CORE

RISC-V SoC • RTL-to-GDSII



TECHNOLOGY

IHP SG13G2

130 nm CMOS



LOGIC + MEMORY

3,881 standard cells

1 × SRAM macro

1024 × 32 bits

SRAM counted separately

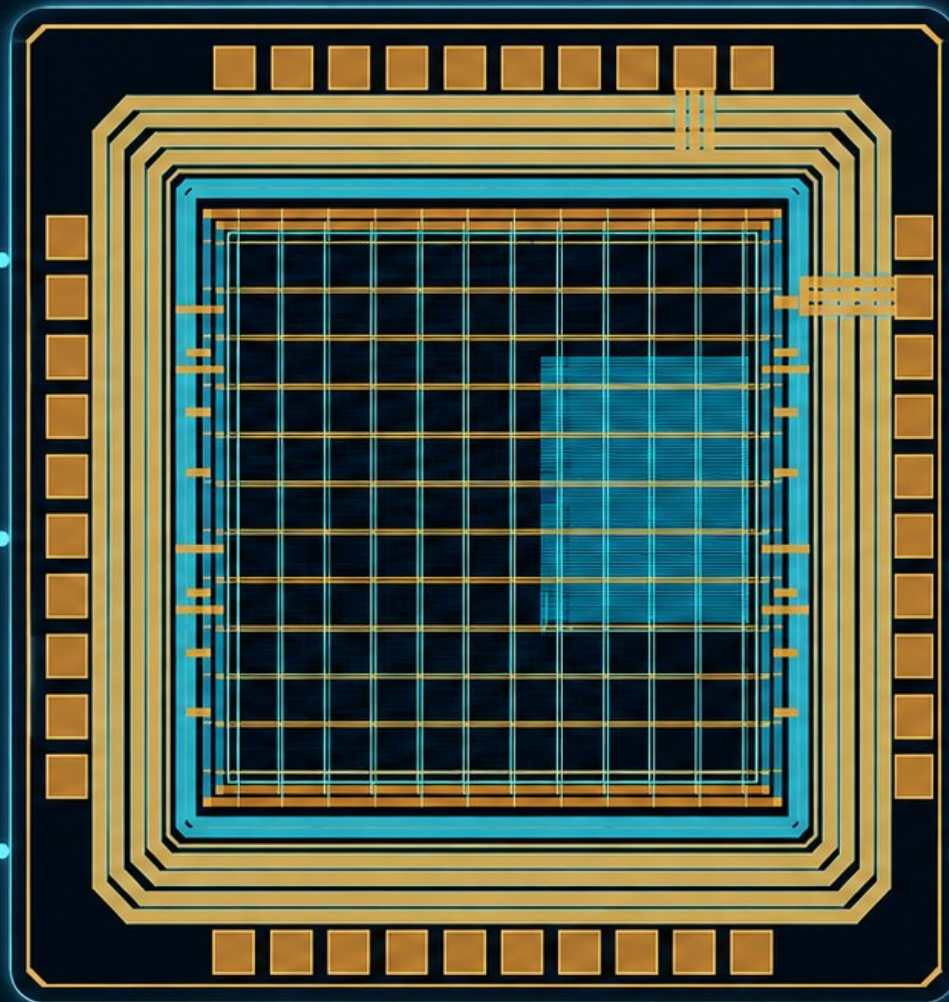


POWER

3.43 mW

Estimated post-layout

Switching: 3.32 mW



DIE + CORE AREA

Die: 1.60 × 1.60 mm

Core: 0.752 mm²



CORE UTILIZATION

26.32%

Std-cell region: 9.45%



CLOCK / TIMING

Closed at 3 PVT corners

Setup margin: +1.11 ns

Hold margin: +0.125 ns

Clock rate not reported



0

SETUP/HOLD
VIOLATIONS



0

ROUTE DRC



0

ANTENNA
VIOLATIONS



WORST IR DROP

0.653 mV



Remaining checks: 13 slew • 32 fanout • 2 density

From Concept to Hardware — The Complete Journey

- Started with a from-scratch, single-cycle RV32I RISC-V core
- Verified against the full ISA, then made GCC-compatible
- Integrated into a complete SoC over a Wishbone bus
- Deployed and proven running on real FPGA hardware
- Carried through a full open-source ASIC flow to a signed-off GDSII layout



LeNet-5 CNN Hardware Accelerator on Zynq-7020

The Project

- **A trained neural network turned into a physical circuit** on FPGA fabric
- **Hand-written HLS** — C++ to RTL, not a vendor IP flow
- **Target:** Xilinx Zynq-7020 (XC7Z020, CLG400)
- **Accelerator in the PL**, ARM Cortex-A9 drives it over AXI-Lite

Headline Results

98.97%

FP32 test accuracy

3.08×

latency speedup

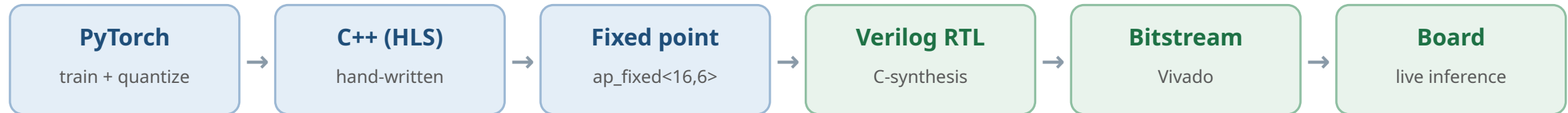
4.65 ms

per inference @ 100 MHz

2.8%

of chip logic used

Design Flow



Every stage was verified against the one before it — the hardware provably computes the same thing as the trained model.

Every Stage — Tool, Input and Output

Stage	Tool used	Input	Output
1 Train	PyTorch (Python)	MNIST — 60,000 training images	Trained weights · 98.97% accuracy
2 Quantize	PyTorch quantization	FP32 trained model	INT8 ONNX · 98.90% accuracy
3 Fit check	Python + Xilinx DS190	Model complexity profile	Verdict: fits on-chip (46.6% BRAM)
4 HLS rewrite	Hand-written C++	Trained weights + architecture	lenet5.cpp — synthesizable C++
5 Fixed point	Vivado HLS ap_fixed<16,6>	FP32 C++ model	16-bit design · 19/20 verified
6 Optimize	Vivado HLS pragmas	1,433,457 cycles	464,699 cycles (3.08× faster)
7 C-synthesis	Vivado HLS 2019.1	lenet5.cpp	Verilog RTL — 25 modules
8 Package IP	Vivado HLS export_design	Verilog RTL	IP core + generated C driver
9 System build	Vivado 2019.1	IP core + Zynq PS + AXI	lenet5_bd_wrapper.bit (3.9 MB)
10 ARM software	Xilinx SDK 2019.1	lenet5_hw.hdf + main.c	executable.elf (182 KB)
11 Deploy	HW Manager + xsct	Bitstream + ELF	Predicted digit printed over UART

Two files leave this pipeline. A 3.9 MB bitstream that configures the FPGA fabric, and a 182 KB executable that runs on the ARM core. Together they run on any machine with a JTAG cable — no project, no source, no rebuild.

Model Training & Hardware Compatibility

LeNet-5 — Trained in PyTorch on MNIST

- **Architecture:** Conv1(1→6) → Pool → Conv2(6→16) → Pool → FC 400-120-84-10
- **10 epochs on MNIST** — 98.97% FP32 test accuracy
- **Chosen over** SqueezeNet, MobileNet and ResNet-18
- Plain sequential convolutions are easiest to hand-pipeline in HLS
- **61,706 parameters** · 418,096 MACs · 291.96 KB memory

Why This Decision Came First

- **If it had not fit**, weights would stream from DDR3 during inference
- **That means tiling, DMA and buffering** — a much harder HLS design
- **Fitting on-chip** let every weight live permanently in BRAM

Does It Fit on the Chip?

Resource	Model needs	Chip has	Used
On-chip BRAM	291.96 KB	627.2 KB	46.6%
DSP48E1	418K MACs	220 slices	trivial
LUTs	—	53,200	—
Flip-flops	—	106,400	—

Verdict: Compatible — fits entirely on-chip.

No DDR streaming or tiling needed. This one result simplified the entire hardware design.

Resource figures confirmed against Xilinx DS190. The generic 7-series selection guides do not cover Zynq parts at all.

Live Demonstration — Input and Output Defined

```

MNIST test image #4920      PyTorch FP32 reference model (server CPU)
+-----+
|                                     |
|                                     |
|          oo..                     |
|      ....00000000000000000000::  |
|  ::##00000000000000000000####00##|
|00000000000000000000000000000000|
|00000000000000000000000000000000|
|ooo00000000000000000000000000000|
|                                     |
|          :.:000000                |
|          ..000000                |
|          00000000                |
|          00000000                |
|          oo000#000000..oo000000..|
|..00000000000000000000000000000000|
|00000000000000000000000000000000..|
|000000## 000000000000000000000000..|
|oo00000#oo000000000000..oo##00000000|
|00000000000000000000000000000000..|
|000000000000..  oo#####000000000000|
|000000000000..  .....              |
|..####:                             |
|                                     |
+-----+

Output scores from layer FC3 (raw logits):

0 ##### -18.70
1 ##### 0.44
2 ##### 57.52 <== PREDICTED
3 ##### -7.10
4 ### -34.65
5 # -41.59
6 -42.95
7 ##### 8.53
8 ##### -11.09
9 # -39.07

=====
PREDICTED DIGIT: 2      true label: 2      CORRECT
margin over next-best (digit 7): 48.99
software inference: 27.44 ms
=====

```

Input

One 28×28 grayscale MNIST test image — 784 pixel values.

Held-out data: these digits were never seen during training. The true label is stored separately and is never passed to the network.

Output

Ten raw scores from the final layer, FC3 — one per digit class.

The largest score is the prediction. The gap to the next-best score shows how confident the network is.

Result Shown

Test image	Predicted	True label	Margin
#4920	2	2 ✓	48.99

The digit is drawn in the terminal from the same 784 values that are fed to the network. Nothing is generated — the image is retrieved from the test set.

Fixed-Point Conversion — FP32 to ap_fixed<16,6>

Why Quantize?

- **Floating point is expensive on FPGA fabric** — more logic, slower
- **ap_fixed<16,6>** — 16 bits total, 6 integer, 10 fractional
- **Values snap to multiples of 1/1024** — nothing exists in between
- **The cost:** a rounding error on every weight, compounding across layers
- **Why it still works:** the prediction is an argmax — only the largest score matters

One Model, Two Implementations

- **The network was trained once**, in PyTorch — there is no second training
- **The C++ was hand-written** and uses those same trained weights
- **Only the arithmetic changed** — both run and agree

Demonstrated live: the same digit through PyTorch and through the compiled fixed-point C++ — the file that was synthesized into the FPGA circuit.

Verification & Resource Impact

Metric	FP32	ap_fixed<16,6>
Predictions matched	reference	19 / 20
BRAM_18K blocks	201 / 280	76 / 280
BRAM utilization	71.8%	27.1%

The one disagreement was not a defect.

On sample 8, PyTorch's own FP32 model predicted the wrong digit. The fixed-point hardware model predicted correctly.

HLS Optimization — Pipelining the Design

What Worked

- **PIPELINE II=1** — a new loop iteration starts every clock cycle
- **Without it**, each iteration finished before the next began
- **Two pragma lines** on the convolution layers gave 2.46×
- **Same treatment on the FC layers** — pipelined end to end
- **No change to the maths** — correctness re-verified at every step

Latency Reduction

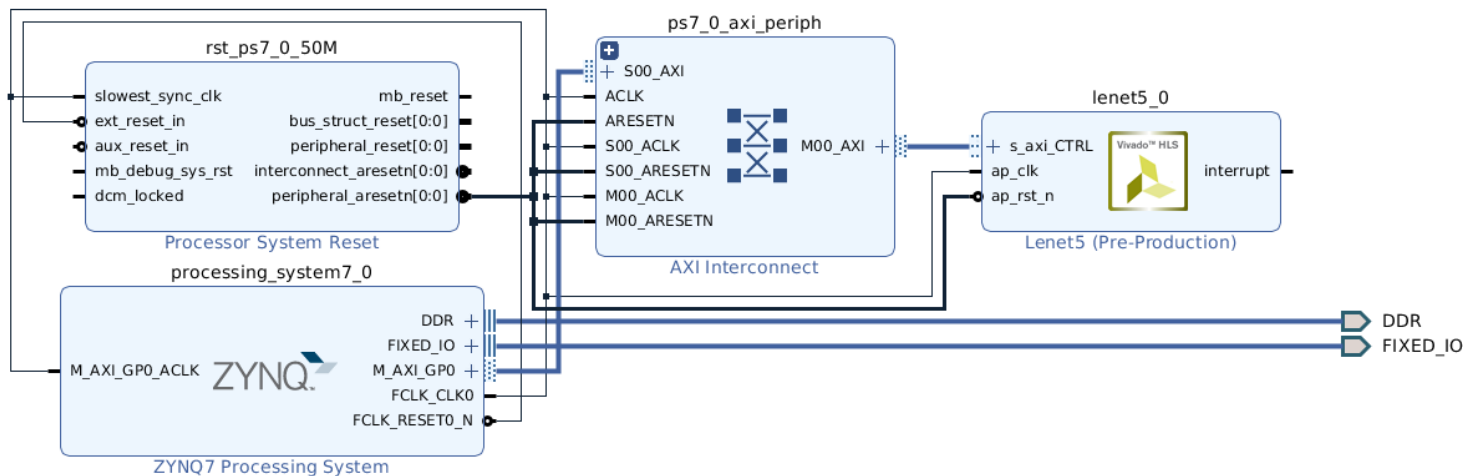
Stage	Cycles	@ 100 MHz
No pragmas (baseline)	1,433,457	14.33 ms
conv1 / conv2 pipelined	582,111	5.82 ms
+ all FC layers pipelined	464,699	4.65 ms

3.08× speedup, DSP usage unchanged at 5 of 220

What Failed — and Why That Matters

- **Full UNROLL**: faster at 366,286 cycles, but violated timing (11.36 ns vs 10 ns) and pushed DSP usage to 81%
- **Root cause**: BRAM has only two read ports — the unrolled loop needed many simultaneous reads
- **ARRAY_PARTITION fixed the ports** but caused a scheduler blowup — one run exceeded five hours with no result
- **Decision**: reverted to the proven pipeline-only design. Reliable beats marginally faster.

System Integration & Implementation Results



Vivado block design — ZYNQ7 Processing System, AXI Interconnect and the custom lenet5 IP at 0x43C0_0000

On the Board

- **Programmed over JTAG** — Vivado Hardware Manager loads the bitstream
- **xsct loads the ARM program**, which prints the predicted digit over UART
- **HLS predicted BRAM and DSP exactly**, but overestimated LUTs by 3.6×

Pipelining over unrolling kept the DSP count low. 97% of both DSPs and LUTs remain unused.

Placed & Routed on Silicon

Resource	Used	Util.
Slice LUTs	1,481	2.78%
Slice Registers	1,483	1.39%
Block RAM	40 / 140	28.6%

5 of 220 DSPs

Timing closed with 11.8 ns slack against a 10 ns target — zero failing endpoints out of 4,535.

The accelerator itself: 1,083 LUTs — 73% of the system logic, and all of its BRAM and DSPs.

Thank You

Questions & Live Demonstration

LeNet-5 on Xilinx Zynq-7020 · Hand-written HLS accelerator